

City Under Attack: Sanctuary Defense

Game survival pasca-apokaliptik yang dikembangkan dengan **Pygame** dan **Computer Vision (MediaPipe)** untuk kontrol interaktif.

Fitur Utama

- **Top-Down Exploration:** Jelajahi "The Sanctuary", benteng terakhir kemanusiaan.
- **Dynamic NPC Interaction:** Berinteraksi dengan warga dan pemimpin kota.
- **Tactical Shooting Mode:** Mode "City Under Attack" di mana pemain harus bertahan dari serangan zombie.
- **Gesture Control (Computer Vision):** Gunakan gerakan tangan untuk menggerakkan karakter dan menembak (PISTOL gesture).

Struktur Proyek

- `main.py`: Entry point dan main game loop.
- `src/core/`:
- `sanctuary_logic.py`: Logika dunia, NPC, dan transisi ke pertempuran.
- `minigame_manager.py`: Mengelola pergantian ke mode shooting.
- `camera.py` & `visual_effects.py`: Sistem kamera dan efek visual.
- `src/entities/`: Kelas untuk Player dan NPC.
- `src/ui/`: Sistem HUD dan Dialog Box.
- `comvis/`: Sistem pengenalan gestur menggunakan MediaPipe.

Cara Menjalankan

1. Install dependensi: `pip install -r requirements.txt`

2. Jalankan game: `python main.py`

Kontrol

- **Keyboard:** Panah (Gerak), Enter (Interaksi), Shift (Lari).
- **Gestur:**
- Gerakkan tangan untuk mengontrol kursor/karakter.
- Gestur "PISTOL" (Telunjuk ke atas) + sentakan ke atas untuk menembak.

■ MENU DOKUMENTASI & PENJELASAN KODE (LINE-BY-LINE)

Halo! Karena proyek ini akan didemokan, semua berkas penjelasan baris-demi-baris (.md) telah dikumpulkan ke dalam satu folder terpusat yaitu **dokumentasi/** di direktori utama agar mudah diakses.

Gunakan menu di bawah ini untuk langsung membuka dokumen penjelasan untuk setiap berkas kode:

■ 1. Alur Utama Game (Root)

- ■ **Main Entry Point:** [main.md](#) (Menjelaskan inisialisasi game loop, kustomisasi wardrobe, gesture control, dan integrasi minigame)

■ 2. Modul Computer Vision (Gesture Kamera)

- ■ **Gestures Thread Tracker:** [gestures.md](#) (Thread pelacakan landmark tangan MediaPipe & klasifikasi gestur real-time)
- ■ **Visualisasi Logika Pistol:** [visualisasi_logika_pistol.md](#) (Visualisasi geometris dan logika pemicu tembakan recoil)

■■ 3. Inti Logika Game (src/core/)

- ■■ Sanctuary Logic: [sanctuary_logic.md](#) (Logika interaksi map, api ungun, kasur medis, helipad drop, loker pakaian)
- ■ Camera Viewport: [camera.md](#) (Mengatur pergerakan kamera mengikuti posisi koordinat pemain)
- ■ Currency Manager: [currency.md](#) (Mengelola koin perunggu/perak/emas dan konversi otomatis)
- ■ Visual Effects (VFX): [visual_effects.md](#) (Menghasilkan senter/gelap, screen shake recoil, dan kilatan cahaya)
- ■ Audio System: [audio_manager.md](#) (Mengontrol pemutaran musik latar BGM dan efek suara SFX)
- ■■ Sub-Game Orchestrator: [minigame_manager.md](#) (Mengatur pergantian state dari game utama ke minigame dan pembagian koin)
- ■ Shooting Minigame: [shooting_range.md](#) (FPS Zombie Shooter, aim-assist lock, progressive multiplier, time extension)

■ 4. Karakter & Entitas (src/entities/)

- ■ Player Character: [player.md](#) (Logika gerak pemain, efek adrenalin, dan render baju-kulit-rambut-topi berlapis)
- ■ Civilian/Guard AI: [npc.md](#) (AI berkeliaran random/wandering untuk Kapten Jaka, Penatua Aris, dll.)
- ■ Zombie NPC: [zombie.md](#) (Perilaku zombie mengejar, terkena taser/molotov/decoy)
- ■ Loot/Bronze Drops: [loot.md](#) (Deteksi jarak dan animasi melayang koin jarahan)
- ■ Tactical Throwables: [tactical_item.md](#) (Fisika lemparan Molotov, Decoy, dan Taser)

■■ 5. Antarmuka Layar (src/ui/)

- **HUD (Heads-Up Display):** [hud.md](#) (Menggambar bar darah, baterai senter, dan animasi angka koin)

- **Dialogue System:** [dialogue.md](#) (Menampilkan kotak dialog narasi cerita)

Tips: Anda dapat membuka folder dokumentasi/ untuk melihat semua file secara langsung, atau mengeklik tautan di atas jika Anda membukanya di VS Code / GitHub.

Penjelasan Kode: dialogue.py

Dokumen ini berisi penjelasan detail baris demi baris untuk file `/home/abuyyy/PemogramanLanjut/src/ui/dialogue.py`. Penjelasan ini dibuat dalam bahasa Indonesia untuk membantu persiapan demo program.

Deskripsi & Tujuan File

File `dialogue.py` mendefinisikan kelas `DialogueBox`. Kelas ini berfungsi untuk mengelola dan menampilkan kotak dialog (dialogue box) interaktif dalam permainan berbasis Pygame. Kotak dialog ini mendukung penampilan pesan bertahap (satu per satu), penutupan otomatis setelah pesan terakhir, indikator visual untuk kelanjutan dialog ([Press Enter]), serta status khusus "berpikir" (`is_thinking`) yang menampilkan teks dinamis berkedip ("Sedang berpikir...").

Penjelasan Baris Demi Baris

Berikut adalah tabel penjelasan baris demi baris kode di dalam `dialogue.py`:

Baris	Kode Sumber	Penjelasan
1	<code>import pygame</code>	Mengimpor modul pygame agar semua fitur grafis, teks, dan event Pygame dapat digunakan di dalam berkas ini.
2	<code>class DialogueBox:</code>	Deklarasi kelas DialogueBox untuk mengelompokkan data dan logika pengelolaan kotak dialog.
3	<code>def __init__(self, font_size=18):</code>	Konstruktor kelas DialogueBox dengan parameter opsional font_size yang memiliki nilai default 18.
4	<code>self.font = pygame.font.SysFont("monospace", font_size)</code>	Membuat dan memuat objek font sistem berjenis monospace dengan ukuran sesuai parameter font_size.
5	<code>self.active = False</code>	Menentukan status keaktifan kotak dialog. Jika False, kotak dialog tidak akan digambar di layar.
6	<code>self.messages = []</code>	Inisialisasi daftar kosong (list) untuk menyimpan pesan-pesan teks yang akan ditampilkan.
7	<code>self.current_index = 0</code>	Indeks penunjuk pesan yang saat ini sedang aktif (dimulai dari indeks 0 untuk pesan pertama).
8	<code>self.box_rect = pygame.Rect(50, 450, 700, 120)</code>	Mendefinisikan objek Rect Pygame untuk menentukan batas kotak dialog: posisi X=50, Y=450, lebar=700, dan tinggi=120.

9	<code>self.is_thinking = False</code>	Menambahkan variabel status berpikir untuk menandai apakah sistem sedang memproses sesuatu (misalnya memikirkan jawaban AI).
10	<code>def show(self, messages, is_thinking=False):</code>	Mendefinisikan metode show untuk mengaktifkan kotak dialog dengan pesan tertentu dan status berpikir opsional.
11	<code>if isinstance(messages, str):</code>	Memeriksa apakah parameter messages bertipe string tunggal (bukan list).
12	<code>self.messages = [messages]</code>	Jika berupa string, bungkus string tersebut ke dalam list berisi satu elemen agar struktur data tetap konsisten.
13	<code>else:</code>	Blok alternatif jika parameter messages bukan string tunggal (melainkan list/koleksi string).
14	<code>self.messages = messages</code>	Menyimpan langsung list pesan ke dalam variabel <code>self.messages</code> .
15	<code>self.current_index = 0</code>	Mengatur ulang penunjuk indeks ke 0 untuk memastikan dialog dimulai dari pesan pertama.
16	<code>self.active = True</code>	Mengubah status kotak dialog menjadi aktif (True) agar mulai ditampilkan di layar.
17	<code>self.is_thinking = is_thinking</code>	Menyetel nilai status berpikir sesuai dengan nilai parameter <code>is_thinking</code> yang dikirim.
18	<code>def next_message(self):</code>	Mendefinisikan metode <code>next_message</code> untuk berlanjut ke pesan berikutnya dalam antrian.
19	<code>self.current_index += 1</code>	Meningkatkan indeks pesan aktif sebesar 1 untuk bergeser ke pesan selanjutnya.
20	<code>if self.current_index >= len(self.messages):</code>	Memeriksa apakah indeks saat ini sudah sama atau melebihi jumlah pesan dalam list.
21	<code>self.close()</code>	Jika pesan sudah habis, panggil metode <code>close()</code> untuk menutup kotak dialog.
22	<code>return False</code>	Mengembalikan False untuk memberi tahu pemanggil bahwa dialog telah selesai/ditutup.
23	<code>return True</code>	Mengembalikan True jika masih ada pesan selanjutnya yang berhasil ditampilkan.
24	<code>def close(self):</code>	Mendefinisikan metode <code>close</code> untuk menutup kotak dialog secara manual.
25	<code>self.active = False</code>	Mengubah status keaktifan kotak dialog menjadi tidak aktif (False).
26	<code>self.messages = []</code>	Mengosongkan daftar pesan yang sebelumnya tersimpan.
27	<code>self.current_index = 0</code>	Mengatur ulang indeks pesan aktif kembali ke 0.
28	<code>def draw(self, screen):</code>	Mendefinisikan metode <code>draw</code> untuk menggambar seluruh elemen visual kotak dialog pada permukaan <code>screen</code> .
29	<code>if self.active and self.messages:</code>	Menggambar hanya jika kotak dialog aktif (<code>active</code> bernilai True) dan list pesan tidak kosong.

30	<code>shadow_rect = self.box_rect.copy()</code>	Membuat salinan (duplikat) dari koordinat kotak dialog <code>self.box_rect</code> .
31	<code>shadow_rect.x += 4</code>	Menggeser posisi X bayangan sejauh 4 piksel ke kanan.
32	<code>shadow_rect.y += 4</code>	Menggeser posisi Y bayangan sejauh 4 piksel ke bawah.
33	<code>pygame.draw.rect(screen, (20, 20, 20), shadow_rect)</code>	Menggambar bayangan kotak dengan warna abu-abu sangat gelap (RGB: 20, 20, 20) menggunakan objek <code>shadow_rect</code> .
34	<code>pygame.draw.rect(screen, (40, 40, 60), self.box_rect)</code>	Menggambar latar belakang kotak utama dengan warna biru tua keabu-abuan (RGB: 40, 40, 60).
35	<code>pygame.draw.rect(screen, (180, 180, 200), self.box_rect, 3)</code>	Menggambar garis tepi kotak utama setebal 3 piksel dengan warna abu-abu muda (RGB: 180, 180, 200).
36	<code>if self.is_thinking:</code>	Memeriksa apakah kotak dialog berada dalam status sedang berpikir (<code>self.is_thinking</code> bernilai True).
37	<code>pulse = (pygame.time.get_ticks() // 500) % 2</code>	Mengambil waktu aktif sistem (dalam milidetik), dibagi 500, lalu di-modulo 2 untuk mendapatkan nilai 0 atau 1 yang bergantian setiap 0.5 detik.
38	<code>text = "Sedang berpikir" + ("..." if pulse else ".")</code>	Membuat efek animasi titik-titik berkedip. Teks akan bertuliskan "Sedang berpikir..." jika <code>pulse == 1</code> , dan "Sedang berpikir." jika <code>pulse == 0</code> .
39	<code>else:</code>	Blok alternatif jika kotak dialog menampilkan pesan teks biasa.
40	<code>text = self.messages[self.current_index]</code>	Mengambil teks pesan aktif saat ini berdasarkan indeks <code>self.current_index</code> .
41	<code>text_surface = self.font.render(text, True, (255, 255, 255))</code>	Merender teks menjadi permukaan gambar grafis (Surface) berwarna putih (RGB: 255, 255, 255) dengan fitur <i>anti-aliasing</i> (True).
42	<code>screen.blit(text_surface, (self.box_rect.x + 20, self.box_rect.y + 25))</code>	Menggambar permukaan teks ke layar dengan posisi relatif di dalam kotak dialog (ditambah padding kiri sebesar 20 piksel dan padding atas 25 piksel).
43	<code>if not self.is_thinking:</code>	Petunjuk kelanjutan hanya ditampilkan jika kotak dialog sedang tidak dalam kondisi berpikir (<code>is_thinking == False</code>).
44	<code>hint = self.font.render("[Press Enter]", True, (200, 200, 100))</code>	Merender teks petunjuk "[Press Enter]" dengan warna kuning redup (RGB: 200, 200, 100).
45	<code>screen.blit(hint, (self.box_rect.right - 150, self.box_rect.bottom - 35))</code>	Menempelkan teks petunjuk tersebut di pojok kanan bawah kotak dialog (150 piksel dari sisi kanan kotak, dan 35 piksel dari sisi bawah kotak).
46	<code>(Baris Kosong)</code>	Akhir dari berkas kode sumber.

Alur Kerja Utama

Berikut adalah alur kerja dasar dari objek `DialogBox` selama permainan berlangsung:

```
flowchart TD
    A[DialogBox Dibuat __init__] --> B[Menerima pesan lewat show]
    B --> C{Apakah is_thinking True?}
    C -- Ya --> D[Tampilkan 'Sedang berpikir...' berkedip]
    C -- Tidak --> E[Tampilkan pesan indeks ke-0]
    E --> F[Tampilkan petunjuk Press Enter]
    F --> G[Pemain menekan Enter / Kejadian Pemicu]
    G --> H[Panggil next_message]
    H --> I{Apakah ada pesan selanjutnya?}
    I -- Ya --> E
    I -- Tidak --> J[Panggil close & kotak dialog menghilang]
```

1. **Inisialisasi:** Kotak dialog dibuat dengan font monospace default ukuran 18, berposisi di bagian bawah layar game ($y=450$, tinggi=120, lebar=700). Keadaan default-nya adalah tidak aktif (`active = False`).
2. **Aktivasi (show):** Kode eksternal memanggil `.show(["Pesan 1", "Pesan 2"])`. Ini langsung mereset indeks ke 0 dan mengeset `active = True`.
3. **Menggambar (draw):** Di dalam *game loop*, jika statusnya aktif, kotak dialog akan digambar ke permukaan layar (background, border, bayangan, teks pesan saat ini, dan tombol instruksi kelanjutan).
4. **Navigasi (next_message):** Ketika pemain menekan tombol Enter (ditangani di *main game loop*), aplikasi memanggil `.next_message()`. Indeks akan bertambah. Jika indeks melebihi jumlah pesan, kotak akan memanggil `.close()` secara otomatis untuk bersembunyi.

Penjelasan Kode: engine.py

Dokumen ini berisi penjelasan detail baris demi baris untuk file `/home/abuyyy/PemogramanLanjut/src/core/engine.py`. Penjelasan ini ditulis dalam bahasa Indonesia untuk membantu persiapan demo program.

Deskripsi & Tujuan File

File `engine.py` berfungsi sebagai penggerak dasar (*core game engine*) permainan. Di dalamnya terdapat beberapa kelas utilitas penting:

1. **Spritesheet:** Memotong-motong lembaran gambar sprite besar (*spritesheet*) menjadi frame individual berskala untuk dianimasikan.
2. **Camera:** Mengatur efek pergeseran kamera 2D scroll mengikuti posisi target pemain agar tidak keluar dari area batas peta.
3. **CurrencyManager:** Menyimpan dan melacak saldo keuangan pemain, menangani transaksi penambahan koin perunggu (*Bronze*), serta konversi otomatis pecahan 100 Bronze = 1 Silver, dan 100 Silver = 1 Gold.
4. **AudioManager:** Mengontrol pemutaran musik latar (BGM) secara berulang dan efek suara tembakan/ledakan (SFX).
5. **VisualEffects:** Menggambar lapisan kegelapan malam dengan senter melingkar di sekeliling pemain, efek getaran layar (*screen shake*), dan efek kedipan putih kilatan cahaya tembakan.
6. **MiniGame & MiniGameManager:** Kelas abstraksi dan pengelola alur transisi daur hidup minigame, menyimpan/mengembalikan status eksplorasi, serta membagikan bonus uang perunggu berdasarkan performa bermain.

Penjelasan Baris Demi Baris

Berikut adalah tabel penjelasan baris demi baris dari kode sumber `engine.py`:

Baris	Kode Sumber	Penjelasan
1	<code>import pygame, random, math, os</code>	Mengimpor Pygame untuk grafik/audio, random untuk acak, math untuk trigonometri, dan os untuk path file.
2	<code>from typing import Optional</code>	Mengimpor tipe Optional untuk anotasi tipe data statis.
3	<code>(Baris Kosong)</code>	Pemisah visual antarkelas.
4	<code>class Spritesheet:</code>	Deklarasi kelas Spritesheet untuk memotong dan mengelola kumpulan frame animasi gambar.
5	<code>def __init__(self, filename, cols, rows, scale=1.5):</code>	Konstruktor menerima path file gambar, jumlah kolom, jumlah baris, dan faktor skala pembesaran.
6	<code>self.sheet = pygame.image.load(filename).convert_alpha()</code>	Memuat gambar spritesheet dari path dan mengonversi format warna alpha transparansi piksel.
7	<code>self.cols, self.rows, self.scale = cols, rows, scale</code>	Menyimpan jumlah kolom, jumlah baris, dan skala pembesaran di atribut objek.
8	<code>self.fw, self.fh = self.sheet.get_width() // cols, self.sheet.get_height() // rows</code>	Menghitung lebar (fw) dan tinggi (fh) masing-masing frame gambar (lebar total / kolom, tinggi total / baris).
9	<code>self.frames = [[pygame.transform.scale(self.sheet.subsurface(0, 0, self.fw, r*self.fh, self.fw, self.fh)), (int(self.fw*scale), int(self.fh*scale))] for c in range(cols)] for r in range(rows)]</code>	Menyusun frame individual menggunakan subsurface di setiap koordinat baris/kolom, lalu menskalakan ukurannya secara dinamis menggunakan list comprehension bersarang.
10	<code>def get_frame(self, row, col): return self.frames[row][col]</code>	Metode pembantu untuk mengambil gambar frame tertentu pada baris dan kolom yang diminta.
11	<code>(Baris Kosong)</code>	Pemisah visual antarkelas.
12	<code>class Camera:</code>	Deklarasi kelas Camera untuk mengendalikan pandangan scroll 2D.
13	<code>def __init__(self, w, h, mw, mh):</code>	Konstruktor menerima lebar layar (w), tinggi layar (h), lebar peta maksimal (mw), dan tinggi peta maksimal (mh).
14	<code>self.camera = pygame.Rect(0, 0, w, h)</code>	Membuat objek Rect Pygame sebagai pembatas pandangan kamera awal di (0,0).
15	<code>self.w, self.h, self.mw, self.mh = w, h, mw, mh</code>	Menyimpan resolusi layar dan ukuran maksimal peta.
16	<code>def apply(self, pos):</code>	Menghitung posisi objek di layar berdasarkan posisi aslinya di peta dikurangi pergeseran kamera.
17	<code>return pos.move(self.camera.topleft) if isinstance(pos, pygame.Rect) else (pos[0] + self.camera.x, pos[1] + self.camera.y)</code>	Jika bertipe Rect, geser topleft-nya. Jika berupa koordinat list/tuple, tambahkan offset X/Y kamera.
18	<code>def update(self, t):</code>	Metode untuk memperbarui koordinat kamera agar berpusat mengikuti target posisi t (pemain).
19	<code>x = max(-(self.mw - self.w), min(0, -t[0] + self.w // 2))</code>	Menghitung posisi horizontal X kamera agar berada di tengah target, dibatasi agar tidak melampaui batas tepi peta.

20	<code>y = max(-(self.mh - self.h), min(0, -t[1] + self.h // 2))</code>	Menghitung posisi vertikal Y kamera agar berada di tengah target, dibatasi batas tepi peta.
21	<code>self.camera = pygame.Rect(x, y, self.w, self.h)</code>	Menyimpan koordinat kamera baru ke dalam objek Rect
22	<i>(Baris Kosong)</i>	self.camera.
23	<code>class CurrencyManager:</code>	Pemisah visual antarkelas. Deklarasi kelas
24	<code>def __init__(self): self.bronze = self.silver = self.gold = 0</code>	CurrencyManager untuk mengelola uang koin pemain. Konstruktor menetapkan saldo awal bronze, silver, dan gold ke 0.
25	<code>def add_bronze(self, amt): self.bronze += amt; self._convert()</code>	Menambahkan sejumlah uang perunggu (bronze) lalu memicu fungsi konversi nilai mata uang.
26	<code>def get_total_bronze(self): return self.bronze + (self.silver * 100) + (self.gold * 10000)</code>	Mengalkulasi total seluruh uang jika semua koin dilebur ke satuan perunggu (1 Silver = 100 Bronze, 1 Gold = 10.000 Bronze).
27	<code>def deduct_bronze(self, amt):</code>	Mengurangi saldo perunggu pemain jika total seluruh uang mencukupi.
28	<code>tot = self.get_total_bronze()</code>	Mendapatkan total kekayaan pemain dalam satuan perunggu.
29	<code>if tot < amt: return False</code>	Jika total uang kurang dari jumlah yang ditarik, batalkan transaksi dan kembalikan False.
30	<code>self.gold, r = divmod(tot - amt, 10000)</code>	Menghitung sisa koin emas baru dengan membagi total sisa uang dengan 10.000 (fungsi divmod mengembalikan hasil bagi dan sisa).
31	<code>self.silver, self.bronze = divmod(r, 100)</code>	Menghitung sisa koin perak dan perunggu dari sisa kembalian pembagian sebelumnya dibagi 100.
32	<code>return True</code>	Mengembalikan True tanda pengurangan sukses dilakukan.
33	<code>def _convert(self):</code>	Metode internal mengonversi pecahan perunggu ke perak dan perak ke emas.
34	<code>if self.bronze >= 100: self.silver += self.bronze // 100; self.bronze %= 100</code>	Jika koin perunggu >= 100, konversi kelipatannya ke perak dan simpan sisanya.
35	<code>if self.silver >= 100: self.gold += self.silver // 100; self.silver %= 100</code>	Jika koin perak >= 100, konversi kelipatannya ke emas dan simpan sisanya.
36	<i>(Baris Kosong)</i>	Pemisah visual antarkelas.
37	<code>class AudioManager:</code>	Deklarasi kelas AudioManager untuk memproses musik dan efek suara.
38	<code>def __init__(self):</code>	Konstruktor pembuat manajer audio.
39	<code>pygame.mixer.init()</code>	Menginisialisasi modul audio mixer milik Pygame.
40	<code>self.bgm_volume, self.sfx_volume, self.current_bgm = 0.5, 0.7, None</code>	Mengeset volume musik latar default (50%), volume efek suara (70%), dan lagu latar aktif saat ini ke None.
41	<code>def play_bgm(self, f, loops=-1):</code>	Memutar file musik latar (BGM) secara looping terus menerus.
42	<code>try:</code>	Blok penanganan kesalahan pemuatan audio.

43	if self.current_bgm == f: return	Jika lagu latar yang diminta sedang diputar saat ini, jangan putar ulang agar tidak lag.
44	pygame.mixer.music.load(f)	Memuat file audio musik latar dari path f.
45	pygame.mixer.music.set_volume(self.bgm_volume)	Mengatur volume suara pemutaran musik latar.
46	pygame.mixer.music.play(loops)	Memutar musik latar secara berulang sesuai parameter loops (default -1 berarti tiada henti).
47	self.current_bgm = f	Menyimpan nama path file lagu latar aktif saat ini.
48	except: pass	Abaikan kesalahan jika file musik rusak atau tidak ditemukan.
49	def stop_bgm(self): pygame.mixer.music.stop(); self.current_bgm = None	Menghentikan pemutaran musik latar dan mereset current_bgm ke None.
50	def play_sfx(self, f):	Memutar efek suara (SFX) singkat secara instan.
51	try:	Blok penanganan kesalahan pemuatan audio SFX.
52	s = pygame.mixer.Sound(f)	Memuat file efek suara dari path f ke objek Sound.
53	s.set_volume(self.sfx_volume)	Mengatur tingkat volume pemutaran efek suara.
54	s.play()	Memainkan suara efek sekali jalan.
55	except: pass	Abaikan kesalahan jika file suara tidak ditemukan.
56	def set_volumes(self, bv, sv):	Mengubah volume musik latar dan efek suara secara dinamis.
57	self.bgm_volume, self.sfx_volume = bv, sv	Menyimpan setelan volume BGM dan SFX baru ke atribut objek.
58	pygame.mixer.music.set_volume(bv)	Menerapkan volume baru ke lagu latar yang sedang berputar di sistem.
59	(Baris Kosong)	Pemisah visual antarkelas.
60	class VisualEffects:	Deklarasi kelas VisualEffects untuk menggambar efek getaran, gelap malam, dan kedipan tembakan.
61	def __init__(self, w, h):	Konstruktor menerima lebar (w) dan tinggi (h) layar game.
62	self.w, self.h = w, h	Menyimpan dimensi layar game.
63	self.darkness_surf = pygame.Surface((w, h), pygame.SRCALPHA)	Membuat bidang permukaan gambar kegelapan malam seukuran layar berkemampuan transparansi alpha (SRCALPHA).
64	self.flash_surf = pygame.Surface((w, h))	Membuat bidang permukaan gambar kedipan putih kilatan seukuran layar.
65	self.flash_surf.fill((255,255,255))	Mewarnai permukaan kilatan dengan warna putih polos (RGB 255,255,255).
66	self.flash_timer = self.flash_duration = self.shake_amount = 0	Menginisialisasi timer kilatan, durasi kilatan, dan tingkat getaran layar awal ke 0.
67	def draw_darkness(self, screen, p_pos, camera, light_on, bat, injured):	Metode untuk menggambar kegelapan malam dengan pendaran lingkaran senter di tubuh pemain.
68	c = (25 + int(abs(math.sin(pygame.time.get_ticks() * 0.003)) * 40), 15, 15) if injured else (15, 15, 25)	Memilih warna kegelapan: berdenyut kemerahan jika pemain cedera (injured = True), atau biru kehitaman jika sehat.

69	<code>self.darkness_surf.fill((*c, 255))</code>	Mewarnai seluruh bidang kegelapan dengan warna terpilih secara pekat (alpha 255).
70	<code>if light_on and bat > 0:</code>	Jika senter dinyalakan oleh pemain dan daya baterai masih tersisa (> 0).
71	<code>sp = camera.apply(p_pos)</code>	Menghitung koordinat posisi pemain di layar menggunakan kamera.
72	<code>cx, cy = int(sp[0] + 32), int(sp[1] + 32)</code>	Menghitung koordinat titik tengah senter (posisi X/Y pemain + offset 32px setengah ukuran sprite).
73	<code>for r in range(200, 0, -8):</code>	Looping menggambar gradasi lingkaran luar ke dalam dari jejari 200px mengecil kelipatan 8px.
74	<code>pygame.draw.circle(self.darkness_surf, (0,0,0, max(0, min(255, int(255 * (1.0 - (r / 200)))))), (cx, cy), r)</code>	Menggambar lingkaran transparan parsial di atas permukaan kegelapan. Semakin dekat ke pusat lingkaran, nilai alpha semakin mendekati 0 (transparan), membentuk efek pendaran senter.
75	<code>screen.blit(self.darkness_surf, (0, 0), special_flags=pygame.BLEND_RGBA_SUB)</code>	Menempelkan bidang kegelapan ke layar menggunakan operasi masking pengurangan piksel (BLEND_RGBA_SUB).
76	<code>def trigger_flash(self, dur=120): self.flash_timer, self.flash_duration, self.shake_amount = dur, max(1, dur), 15</code>	Memicu getaran layar (15px) dan kedipan putih layar dengan durasi dur frame.
77	<code>def draw_flash(self, screen):</code>	Menggambar efek getaran layar dan kilatan putih pudar secara gradual.
78	<code>off = [random.randint(-self.shake_amount, self.shake_amount), random.randint(-self.shake_amount, self.shake_amount)] if self.shake_amount > 0 else [0, 0]</code>	Menghitung offset koordinat layar acak horizontal/vertikal jika getaran layar aktif (> 0).
79	<code>if self.shake_amount > 0: self.shake_amount -= 1</code>	Meredam kekuatan getaran layar dengan mengurangnya 1 per frame.
80	<code>if self.flash_timer > 0: self.flash_surf.set_alpha(max(0, min(255, int((self.flash_timer / self.flash_duration) * 255))))</code>	Jika timer kilatan kedipan putih sedang berjalan.
81	<code>int((self.flash_timer / self.flash_duration) * 255))))</code>	Mengatur tingkat transparansi alpha kilatan putih agar memudar seiring berkurangnya timer.
82	<code>screen.blit(self.flash_surf, off)</code>	Menggambar kilatan putih di layar dengan tambahan offset koordinat getaran.
83	<code>self.flash_timer -= 1</code>	Mengurangi timer kedipan putih sebanyak 1 per frame.
84	<code>return tuple(off)</code>	Mengembalikan koordinat offset getaran layar untuk disesuaikan pada rendering peta Sanctuary.
85	<code>(Baris Kosong)</code>	Pemisah visual antarkelas.
86	<code>class MiniGame:</code>	Deklarasi kelas induk dasar MiniGame yang diabstraksi untuk membuat minigame baru.

87	<code>def __init__(self, scr,</code>	Konstruktor menerima objek screen
	<code>clk, mgr):</code>	utama, objek clock Pygame, dan
	<code>self.screen, self.clock,</code>	objek minigame manager.
88	<code>self.manager,</code>	Menyimpan referensi screen, clock,
	<code>self.running, self.score =</code>	manager, status keaktifan game
	<code>scr, clk, mgr, True, 0</code>	(True), dan skor awal (0).
89	<code>self.font =</code>	Inisialisasi font default Arial 24px
	<code>pygame.font.SysFont("Arial",</code>	tebal untuk kebutuhan minigame.
	<code>24, bold=True)</code>	
90	<code>def handle_event(self, e):</code>	Metode placeholder untuk
	<code>pass</code>	menangani kejadian/event input di
		minigame (diimplementasikan di
		kelas turunan).
91	<code>def update(self, dt): pass</code>	Metode placeholder logika
		pembaruan frame minigame.
92	<code>def draw(self): pass</code>	Metode placeholder menggambar
		grafis minigame.
93	<code>def exit_game(self,</code>	Metode untuk menghentikan
	<code>res=None): self.running =</code>	minigame dan mengembalikan
	<code>False;</code>	hasil laporan skor ke manager
	<code>self.manager.return_to_main(res)</code>	utama.
94	<code>(Baris Kosong)</code>	Pemisah visual antarkelas.
		Deklarasi kelas
95	<code>class MiniGameManager:</code>	MiniGameManager sebagai
		jembatan daur hidup pemanggilan
		minigame.
96	<code>def __init__(self, eng):</code>	Konstruktor menerima objek proxy
		engine utama game (eng).
	<code>self.main_engine,</code>	Menyimpan referensi engine
97	<code>self.active_game,</code>	utama, objek minigame aktif
	<code>self.saved_main_state =</code>	(None), dan backup status dunia
	<code>eng, None, None</code>	utama (None).
	<code>def start_minigame(self,</code>	
	<code>cls): self.active_game =</code>	Memulai minigame baru dengan
98	<code>cls(self.main_engine.screen,</code>	menginstansiasi kelas minigame
	<code>self.main_engine.clock,</code>	yang diminta (cls).
	<code>self)</code>	
99	<code>def return_to_main(self,</code>	Keluar dari minigame dan
	<code>res):</code>	mengembalikan kontrol ke dunia
		Sanctuary utama.
	<code>self.active_game,</code>	Menghapus instansi minigame aktif
100	<code>self.last_result = None,</code>	dan menyimpan laporan hasil
	<code>res</code>	akhirnya.
		Memberikan bonus uang koin
101	<code>if res: ...</code>	perunggu ke saldo pemain dan
		mengakumulasi skor minigame
		ke total point_kill pada engine
		proxy.
102	<code>def update(self, dt):</code>	Memperbarui logika minigame aktif
		di setiap frame.
	<code>if self.active_game:</code>	Jika sedang bermain minigame,
103	<code>self.active_game.update(dt)</code>	panggil metode .update() minigame tersebut.
		Menggambar visual minigame aktif
104	<code>def draw(self):</code>	ke layar.
		Jika sedang bermain minigame,
105	<code>if self.active_game:</code>	panggil metode .draw() minigame tersebut.
	<code>self.active_game.draw()</code>	
106	<code>@property</code>	Mendeklarasikan properti getter
		untuk mengecek status permainan.
	<code>def in_minigame(self):</code>	Mengembalikan status True jika
107	<code>return self.active_game is</code>	pemain sedang berada di dalam
	<code>not None</code>	minigame.
108	<code>(Baris Kosong)</code>	Akhir dari berkas kode sumber.

Dokumen ini berisi penjelasan detail baris demi baris untuk file `/home/abuyyy/PemogramanLanjut/src/entities/entities.py`.
Penjelasan ini ditulis dalam bahasa Indonesia untuk membantu persiapan demo program.

Deskripsi & Tujuan File

File `entities.py` mendefinisikan kelas-kelas entitas permainan (game entities) yang mendiami dunia Sanctuary maupun area pertempuran:

- 1. **Player**: Karakter utama pemain yang dapat bergerak dengan kontrol keyboard/kemampuan input proxy gestur, memiliki status nyawa, peningkatan kecepatan dari adrenalin, dan menangani render sprite animasi berskala.
- 2. **NPC**: Karakter non-pemain ramah di Sanctuary yang dapat berpatroli secara acak (wandering) di sekitar titik awal spawn, dibatasi jarak maksimum tertentu, dan memiliki animasi sprite pakaian/rambut terpisah.
- 3. **ZombieNPC**: Karakter musuh (zombie) yang mengejar pemain ketika pemain sedang berlari (`state == 'run'`) atau terpancing oleh item taktis Umpan (Decoy). Zombie dapat menerima damage terbakar dari Molotov dan memiliki timer stun.
- 4. **Loot**: Barang jarahan/drop item di tanah (seperti senjata/suplai) yang memiliki animasi melayang naik-turun secara smooth menggunakan fungsi sinus waktu, serta mendeteksi interaksi dengan pemain.
- 5. **TacticalItem**: Objek taktis yang dilempar oleh pemain (seperti Molotov, Decoy, atau Taser) yang bergerak meluncur menuju koordinat target lemparan sebelum aktif memberikan efek area (aoe/stunning) selama durasi tertentu.

Penjelasan Baris Demi Baris

Berikut adalah tabel penjelasan baris demi baris dari kode sumber `entities.py`:

Baris	Kode Sumber	Penjelasan
1	<code>import pygame, math, random</code>	Mengimpor pustaka Pygame untuk rendering grafis, math untuk kalkulasi jarak/trigonometri, dan random untuk nilai acak.
2	<code>from src.core.engine import Spritesheet</code>	Mengimpor kelas utilitas Spritesheet untuk memotong dan menskalakan animasi gambar karakter.
3	<code>(Baris Kosong)</code>	Pemisah visual antarkelas.
4	<code>class Player:</code>	Deklarasi kelas <code>Player</code> untuk mempresentasikan karakter utama pemain.
5	<code>def __init__(self, x, y):</code>	Konstruktur kelas <code>Player</code> dengan parameter posisi awal (<code>x, y</code>).
6	<code>self.pos, self.direction, self.state, self.current_col = [x, y], 'down', 'stand', 0</code> <code>self.sheet = Spritesheet("FREE Mana Seed Character Base Demo 2.0/char_a_p1/char_a_p1_0bas_humn_v00.png", 8, 8, scale=2.0)</code>	Menginisialisasi posisi, arah awal ('down'), kondisi awal ('stand'), dan kolom frame animasi awal ke 0.
7	<code>self.clothing_sheet = Spritesheet("FREE Mana Seed Character Base Demo 2.0/char_a_p1/1out/char_a_p1_1out_fstr_v01.png", 8, 8, scale=2.0)</code>	Memuat spritesheet tubuh dasar manusia untuk pemain dengan skala pembesaran 2.0x.
8	<code>self.clothing_active, self.hair_sheet, self.hat_sheet = True, Spritesheet("FREE Mana Seed Character Base Demo 2.0/char_a_p1/4har/char_a_p1_4har_dap1_v08.png", 8, 8, scale=2.0), None</code>	Memuat spritesheet pakaian default untuk pemain (baju petualang).
9		Mengaktifkan pakaian default, memuat model rambut pemain, dan mengeset topi default ke None.

10	<pre> self.frame_timer, self.anim_speed, self.health, self.injured, self.speed_multiplier, self.adrenaline_timer = 0, 8, 100, True, 0.4, 0 </pre>	Mengeset variabel timer animasi, kecepatan pergantian frame (8 tick), darah awal (100), status terluka awal, pengali kecepatan awal (40% di Sanctuary), dan durasi status adrenalin (0).
11	<pre> def update(self, keys, collision_mask=None, map_size=None): </pre>	Metode untuk memperbarui logika posisi dan animasi pemain berdasarkan tombol keyboard yang ditekan.
12	<pre> ad_bonus = 2.0 if self.adrenaline_timer > 0 else 1.0 </pre>	Menghitung bonus kecepatan 2.0x lipat jika status adrenalin sedang aktif (> 0).
13	<pre> if self.adrenaline_timer > 0: self.adrenaline_timer -= 1 </pre>	Jika timer adrenalin aktif, kurangi nilainya sebanyak 1 frame.
14	<pre> speed = float((5.0 if keys[pygame.K_LSHIFT] else 3.0) * ad_bonus * self.speed_multiplier) </pre>	Menghitung kecepatan gerak: 5.0 jika berlari (K_LSHIFT) atau 3.0 jika berjalan biasa, dikalikan dengan bonus adrenalin dan speed multiplier.
15	<pre> moving = False </pre>	Menetapkan status pergerakan awal pemain adalah tidak bergerak (moving = False).
16	<pre> if keys[pygame.K_RIGHT]: self.pos[0] += speed; self.direction = 'right'; moving = True </pre>	Jika tombol kanan ditekan: tambah koordinat X, ubah arah ke 'right', dan set moving = True.
17	<pre> if keys[pygame.K_LEFT]: self.pos[0] -= speed; self.direction = 'left'; moving = True </pre>	Jika tombol kiri ditekan: kurangi koordinat X, ubah arah ke 'left', dan set moving = True.
18	<pre> if keys[pygame.K_UP]: self.pos[1] -= speed; self.direction = 'up'; moving = True </pre>	Jika tombol atas ditekan: kurangi koordinat Y, ubah arah ke 'up', dan set moving = True.
19	<pre> if keys[pygame.K_DOWN]: self.pos[1] += speed; self.direction = 'down'; moving = True </pre>	Jika tombol bawah ditekan: tambah koordinat Y, ubah arah ke 'down', dan set moving = True.
20	<pre> if map_size: </pre>	Jika parameter ukuran peta dikirimkan (batas tepi peta).
21	<pre> self.pos[0] = max(0, min(self.pos[0], map_size[0] - 32)) </pre>	Membatasi koordinat X agar pemain tidak keluar dari tepi kiri (0) dan tepi kanan peta (lebar peta - 32px).
22	<pre> self.pos[1] = max(0, min(self.pos[1], map_size[1] - 32)) </pre>	Membatasi koordinat Y agar pemain tidak keluar dari tepi atas (0) dan tepi bawah peta (tinggi peta - 32px).
23	<pre> self.state = 'run' if (keys[pygame.K_LSHIFT] and moving) else 'walk' if moving else 'stand' </pre>	Menentukan kondisi gerakan pemain: 'run' jika menekan Shift & bergerak, 'walk' jika hanya bergerak, dan 'stand' jika diam.
24	<pre> self.frame_timer += 1 </pre>	Meningkatkan nilai timer frame animasi untuk pergantian langkah.
25	<pre> if self.frame_timer >= self.anim_speed: </pre>	Jika timer mencapai batas kecepatan animasi yang ditentukan.
26	<pre> self.frame_timer = 0 </pre>	Mengatur ulang timer frame animasi kembali ke 0.

27	<pre> self.current_col = 0 if self.state == 'stand' else (self.current_col + 1) % 6 if self.state == 'walk' else 6 + (self.current_col + 1) % 2 </pre>	Memilih kolom frame sprite: 0 jika diam, indeks 0 s.d 5 bergantian jika berjalan, dan indeks 6 s.d 7 bergantian jika berlari.
28	<pre> def draw(self, screen, camera=None): row = {'down': 0, 'up': 1, 'right': 2, 'left': </pre>	Metode untuk merender gambar sprite pemain ke layar.
29	<pre> 3}[self.direction] + (0 if self.state == 'stand' else 4) dp = </pre>	Menentukan baris pada spritesheet: baris 0-3 untuk diam (sesuai arah), atau ditambah offset 4 (baris 4-7) untuk berjalan/berlari.
30	<pre> camera.apply(self.pos) if camera else self.pos screen.blit(self.sheet.get_frame(row, </pre>	Menghitung posisi rendering di layar disesuaikan dengan posisi kamera scroll.
31	<pre> self.current_col), dp) if self.clothing_active and self.clothing_sheet: </pre>	Menggambar sprite tubuh dasar pemain di posisi render.
32	<pre> screen.blit(self.clothing_sheet.get_frame(row, self.current_col), dp) if self.hair_sheet: </pre>	Menggambar layer sprite pakaian di atas tubuh dasar jika berstatus aktif.
33	<pre> screen.blit(self.hair_sheet.get_frame(row, self.current_col), dp) if self.hat_sheet: </pre>	Menggambar layer sprite rambut pemain di atas tubuh dasar.
34	<pre> screen.blit(self.hat_sheet.get_frame(row, self.current_col), dp) </pre>	Menggambar layer sprite topi/aksesoris kepala jika ada.
35	<pre> (Baris Kosong) </pre>	Pemisah visual antarkelas.
36	<pre> class NPC: </pre>	Deklarasi kelas NPC untuk karakter non-pemain yang ramah di dunia Sanctuary.
37	<pre> def __init__(self, name, x, y, role, base_path, outfit_path=None, hair_path=None): self.name, self.pos, </pre>	Konstruktor kelas NPC menerima nama, koordinat awal, peran, dan path asset spritesheet tubuh, pakaian, dan rambut.
38	<pre> self.role = name, [x, y], role self.sheet = </pre>	Menyimpan nama, list koordinat posisi [x, y], dan peran (role) dari NPC.
39	<pre> Spritesheet(base_path, 8, 8, scale=2.0) self.outfit_sheet = </pre>	Memuat spritesheet tubuh dasar NPC dengan skala 2.0x.
40	<pre> Spritesheet(outfit_path, 8, 8, scale=2.0) if outfit_path else None self.hair_sheet = </pre>	Memuat spritesheet baju NPC jika ada.
41	<pre> Spritesheet(hair_path, 8, 8, scale=2.0) if hair_path else None self.direction, </pre>	Memuat spritesheet rambut NPC jika ada.
42	<pre> self.state, self.current_col, self.frame_timer, self.anim_speed = 'down', 'stand', 0, 0, 8 self.wander_timer, </pre>	Mengatur arah awal ('down'), kondisi awal ('stand'), indeks kolom animasi (0), timer frame (0), dan kecepatan animasi (8).
43	<pre> self.target_dir, self.speed, self.spawn_pos, self.max_wander_dist = random.randint(30, 90), [0, 0], 1.0, [x, y], 100 </pre>	Menginisialisasi waktu tunggu patroli acak, arah tujuan patroli, kecepatan gerak (1.0), posisi asal spawn, dan jarak radius patroli maksimal (100px).
44	<pre> def update(self, map_size=None): </pre>	Metode untuk memperbarui AI pergerakan acak NPC.

45	<code>self.wander_timer -= 1</code>	Mengurangi timer tunggu patroli acak NPC sebanyak 1 per frame.
46	<code>if self.wander_timer <= 0:</code>	Jika timer habis, saatnya mengambil keputusan patroli baru.
47	<code>self.wander_timer = random.randint(60, 180)</code>	Mengatur ulang timer tunggu acak berikutnya antara 60 hingga 180 frame.
48	<code>if random.random() < 0.5:</code>	Peluang 50% untuk memutuskan diam saja di tempat.
49	<code>self.state, self.target_dir = 'stand', [0, 0]</code>	Mengeset kondisi diam dan arah tujuan ke [0, 0].
50	<code>else:</code>	Peluang 50% lainnya untuk memutuskan bergerak berjalan patroli.
51	<code>angle = random.uniform(0, 2 * math.pi)</code>	Menghasilkan sudut pergerakan acak dalam radian (0 hingga 2*pi).
52	<code>self.target_dir = [math.cos(angle), math.sin(angle)]</code>	Menghitung vektor arah X dan Y berdasarkan sudut menggunakan fungsi kosinus dan sinus.
53	<code>self.state = 'walk'</code>	Mengubah kondisi NPC menjadi sedang berjalan ('walk').
54	<code>dx, dy = self.target_dir</code>	Menyalin nilai arah pergerakan X dan Y.
55	<code>self.direction = ('right' if dx > 0 else 'left') if abs(dx) > abs(dy) else ('down' if dy > 0 else 'up')</code>	Menentukan arah hadap NPC ('right', 'left', 'up', 'down') berdasarkan vektor arah pergerakan terbesar.
56	<code>if self.state == 'walk':</code>	Jika kondisi NPC saat ini adalah sedang berjalan.
57	<code>nx, ny = self.pos[0] + self.target_dir[0]*self.speed, self.pos[1] + self.target_dir[1]*self.speed</code>	Menghitung koordinat posisi target baru nx dan ny berdasarkan arah dan kecepatan.
58	<code>if math.hypot(nx - self.spawn_pos[0], ny - self.spawn_pos[1]) < self.max_wander_dist:</code>	Memeriksa jarak dari posisi baru ke posisi spawn awal. Jika masih di dalam batas radius (100px).
59	<code>self.pos = [nx, ny]</code>	Memperbarui posisi NPC ke koordinat baru tersebut.
60	<code>else:</code>	Jika posisi baru berada di luar batas radius patroli.
61	<code>self.state, self.target_dir, self.wander_timer = 'stand', [0, 0], 15</code>	Membatalkan gerakan, diam di tempat, dan set timer keputusan baru ke 15 frame lagi.
62	<code>if map_size:</code>	Jika dibatasi oleh ukuran peta.
63	<code>self.pos[0] = max(0, min(self.pos[0], map_size[0] - 32))</code>	Membatasi posisi X di dalam area peta agar NPC tidak berjalan ke luar peta.
64	<code>self.pos[1] = max(0, min(self.pos[1], map_size[1] - 32))</code>	Membatasi posisi Y di dalam area peta.
65	<code>self.frame_timer += 1</code>	Menambah timer frame animasi NPC.
66	<code>if self.frame_timer >= self.anim_speed:</code>	Jika timer mencapai batas interval animasi.
67	<code>self.frame_timer = 0</code>	Mereset timer frame animasi NPC ke 0.
68	<code>self.current_col = 0 if self.state == 'stand' else (self.current_col + 1) % 6</code>	Mengeset frame: 0 jika diam, atau bergantian kolom 0 s.d 5 jika berjalan.
69	<code>def draw(self, screen, camera=None):</code>	Metode untuk menggambar grafis sprite NPC ke layar.

70	<pre> row = {'down': 0, 'up': 1, 'right': 2, 'left': 3}[self.direction] + (0 if self.state == 'stand' else 4) dp = </pre>	Memilih baris di spritesheet berdasarkan arah dan status (diam vs berjalan).
71	<pre> camera.apply(self.pos) if camera else self.pos </pre>	Menghitung posisi gambar NPC disesuaikan pergerakan kamera.
72	<pre> screen.blit(self.sheet.get_frame(row, self.current_col), dp) </pre>	Menggambar layer sprite tubuh dasar NPC.
73	<pre> if self.outfit_sheet: screen.blit(self.outfit_sheet.get_frame(row, self.current_col), dp) </pre>	Menggambar layer pakaian NPC jika ada.
74	<pre> if self.hair_sheet: screen.blit(self.hair_sheet.get_frame(row, self.current_col), dp) </pre>	Menggambar layer rambut NPC jika ada.
75	<pre> (<i>Baris Kosong</i>) </pre>	Pemisah visual antarkelas.
76	<pre> class ZombieNPC: </pre>	Deklarasi kelas ZombieNPC untuk musuh zombie di peta pertempuran.
77	<pre> def __init__(self, x, y): </pre>	Konstruktor kelas ZombieNPC menerima posisi awal (x, y).
78	<pre> self.pos, self.sheet = [x, y], Spritesheet("assets/enemies/zombie_new.png", 8, 8, scale=2.0) self.direction, self.state, self.current_col, self.frame_timer, </pre>	Menyimpan posisi dan memuat spritesheet aset zombie baru berskala 2.0x.
79	<pre> self.anim_speed, self.speed, self.stun_timer, self.health = 'down', 'stand', 0, 0, 10, 2.5, 0, 100 </pre>	Inisialisasi arah awal ('down'), kondisi ('stand'), indeks kolom frame, timer animasi, kecepatan animasi (10 tick), kecepatan jalan zombie (2.5), durasi stun (0), dan nyawa zombie (100).
80	<pre> def update(self, player_pos, player_state, tactical_items=[]): </pre>	Metode untuk memperbarui perilaku kecerdasan buatan (AI) zombie mengejar pemain/item taktis.
81	<pre> if self.stun_timer > 0: </pre>	Memeriksa apakah zombie sedang ter-stun (misalnya terkena serangan Taser/Molotov).
82	<pre> self.stun_timer -= 1; self.state = 'stand'; return </pre>	Jika ter-stun: kurangi durasi stun, set kondisi diam ('stand'), dan batalkan pergerakan frame ini.
83	<pre> t_pos, agg = player_pos, (player_state == 'run') </pre>	Menetapkan target kejar default ke pemain, dan status agresif aktif (agg = True) jika pemain berlari ('run').
84	<pre> for item in tactical_items: </pre>	Iterasi untuk mendeteksi keberadaan item taktis yang dilempar di peta.
85	<pre> if item.item_type == "Decoy" and item.active and item.reached_target and math.hypot(item.pos[0] - self.pos[0], item.pos[1] - self.pos[1]) < 400: </pre>	Jika ada item Umpan (Decoy) yang aktif, sudah mendarat di target, dan jaraknya dekat (< 400px).
86	<pre> t_pos, agg = item.pos, True; break </pre>	Alihkan target kejaran zombie ke posisi Umpan tersebut, dan set status agresif ke True. Hentikan perulangan.
87	<pre> for item in tactical_items: </pre>	Iterasi kedua untuk mendeteksi interaksi serangan Molotov.

88	<pre> if item.item_type == "Molotov" and item.reached_target and item.active and math.hypot(item.pos[0] - self.pos[0], item.pos[1] - self.pos[1]) < 50: </pre>	Jika zombie berdiri di dalam radius api Molotov (< 50px) yang sedang aktif di tanah.
89	<pre> self.health -= 2; self.stun_timer = 20 </pre>	Kurangi darah zombie sebanyak 2 per frame, dan beri efek stun selama 20 frame.
90	<pre> if agg and self.health > 0: </pre>	Jika zombie dalam kondisi agresif (mengejar target) dan masih hidup.
91	<pre> dx, dy = t_pos[0] - self.pos[0], t_pos[1] - self.pos[1] </pre>	Menghitung jarak horizontal (dx) dan vertikal (dy) ke posisi target.
92	<pre> dist = math.hypot(dx, dy) </pre>	Menghitung jarak lurus ke target.
93	<pre> if dist > 10: </pre>	Jika jarak ke target masih lebih dari 10px.
94	<pre> self.pos[0] += (dx / dist) * self.speed </pre>	Gerakkan zombie mendekati koordinat X target sesuai rasio jarak dan kecepatannya.
95	<pre> self.pos[1] += (dy / dist) * self.speed </pre>	Gerakkan zombie mendekati koordinat Y target.
96	<pre> self.state = 'walk' </pre>	Ubah status kondisi zombie menjadi berjalan ('walk').
97	<pre> self.direction = ('right' if dx > 0 else 'left') if abs(dx) > abs(dy) else ('down' if dy > 0 else 'up') </pre>	Menentukan arah hadap zombie berdasarkan arah pergerakan dominan.
98	<pre> else: self.state = 'stand' </pre>	Jika sangat dekat (< 10px), ubah status menjadi diam di tempat.
99	<pre> else: self.state = 'stand' </pre>	Jika tidak agresif (pemain berjalan pelan/tidak ada decoy), zombie diam di tempat.
100	<pre> self.frame_timer += 1 </pre>	Tambah timer animasi zombie.
101	<pre> if self.frame_timer >= self.anim_speed: </pre>	Jika timer mencapai interval pergantian frame animasi.
102	<pre> self.frame_timer = 0 </pre>	Mereset timer frame ke 0.
103	<pre> self.current_col = 0 if self.state == 'stand' else (self.current_col + 1) % 6 </pre>	Mengganti kolom frame: 0 jika diam, atau 0 s.d 5 secara bergiliran jika berjalan.
104	<pre> def draw(self, screen, camera=None): row = {'down': 0, 'up': 1, 'right': 2, 'left': </pre>	Metode untuk menggambar zombie di layar.
105	<pre> 3}[self.direction] + (0 if self.state == 'stand' else 4) dp = </pre>	Memilih baris spritesheet berdasarkan arah hadap dan status.
106	<pre> camera.apply(self.pos) if camera else self.pos </pre>	Menyesuaikan posisi gambar zombie dengan posisi kamera.
107	<pre> screen.blit(self.sheet.get_frame(row, self.current_col), dp) </pre>	Menggambar sprite zombie ke layar.
108	<pre> (<i>Baris Kosong</i>) </pre>	Pemisah visual antarkelas.
109	<pre> class Loot: </pre>	Deklarasi kelas Loot untuk item jaraan di lantai.
110	<pre> def __init__(self, name, x, y, item_type, image=None, prompt=None): </pre>	Konstruktor menerima nama item, koordinat (x, y), jenis item, gambar opsional, dan petunjuk teks interaksi opsional.
111	<pre> self.name, self.pos, self.item_type = name, (x, y), item_type </pre>	Menyimpan nama, koordinat posisi, dan jenis item loot.

112	<pre> self.prompt, self.collected, self.hover_offset = prompt or f'Tekan [ENTER] untuk ambil {name}", False, 0 </pre>	Menetapkan teks petunjuk interaksi default, status belum diambil (collected = False), dan offset melayang awal (0).
113	<pre> if image: </pre>	Jika gambar item dikirimkan sebagai argumen.
114	<pre> if isinstance(image, str): </pre>	Memeriksa apakah parameter gambar bertipe string (path file gambar).
115	<pre> try: </pre>	Membuka blok penanganan kesalahan saat memuat gambar eksternal.
116	<pre> self.image = pygame.transform.scale(pygame.image.load(image), (64, 64)) </pre>	Memuat gambar dari path, menjaga (image) sebagai parameter, dan menjadi ukuran 64x64px.
117	<pre> except: </pre>	Jika pemuatan gambar dari path gagal/error.
118	<pre> self.image = pygame.Surface((40, 40)); self.image.fill((255, 0, 0)) </pre>	Opsi darurat: buat permukaan merah berukuran 40x40px sebagai pengganti.
119	<pre> else: self.image = pygame.transform.scale(image, (64, 64)) </pre>	Jika parameter merupakan objek Surface Pygame, langsung skala menjadi 64x64px.
120	<pre> else: </pre>	Jika tidak ada parameter gambar yang dikirimkan.
121	<pre> self.image = pygame.Surface((40, 40)); self.image.fill((255, 215, 0)) </pre>	Buat permukaan berwarna emas (255, 215, 0) berukuran 40x40px.
122	<pre> self.rect = self.image.get_rect(topleft=(x, y)) </pre>	Mengambil pembatas persegi panjang (Rect) dari gambar item diposisikan di topleft (x, y).
123	<pre> def check_interaction(self, player_pos): return </pre>	Metode memeriksa apakah pemain berada dekat dengan item loot tersebut.
124	<pre> math.hypot(self.pos[0] - player_pos[0], self.pos[1] - player_pos[1]) < 100 </pre>	Mengembalikan nilai True jika jarak pemain ke posisi item kurang dari 100px.
125	<pre> def get_interaction_prompt(self): return self.prompt </pre>	Mengambil teks pesan instruksi pengambilan item.
126	<pre> def draw(self, screen, player_pos, camera=None): </pre>	Metode untuk menggambar item loot ke layar dengan animasi melayang.
127	<pre> if not self.collected: </pre>	Menggambar item hanya jika statusnya belum diambil oleh pemain.
128	<pre> self.hover_offset = math.sin(pygame.time.get_ticks() * 0.005) * 10 </pre>	Animasi melayang: gunakan fungsi gelombang sinus waktu aktif sistem untuk menghasilkan gerakan naik-turun berkisar +/- 10px.
129	<pre> dp = camera.apply(self.pos) if camera else self.pos </pre>	Menghitung koordinat rendering disesuaikan dengan posisi gulir kamera.
130	<pre> screen.blit(self.image, (dp[0], dp[1] + self.hover_offset)) </pre>	Menggambar gambar item loot di layar dengan tambahan offset melayang naik-turun pada sumbu Y.
131	<pre> (Baris Kosong) </pre>	Pemisah visual antarkelas.
132	<pre> class TacticalItem: </pre>	Deklarasi kelas TacticalItem untuk item taktis yang dapat dilempar (Molotov, Decoy, Taser).

133	def __init__(self, x, y, item_type, target_pos=None):	Konstruktor menerima posisi lempar awal (x, y), jenis item, dan posisi target koordinat pendaratan.
134	self.pos, self.item_type, self.timer, self.active, self.target_pos, self.speed = [x, y], item_type, 0, True, target_pos, 8	Inisialisasi posisi awal, jenis item, timer aktif (0), status aktif (True), posisi target lemparan, dan kecepatan luncur peluru item (8px per frame).
135	self.reached_target = False if target_pos else True	Mengeset status pendaratan: False jika ada koordinat target lempar (harus meluncur dulu), atau True jika tidak ada.
136	def update(self):	Metode untuk memperbarui fisika peluncuran dan durasi aktif item taktis.
137	if not self.reached_target and self.target_pos:	Jika item belum mendarat di target dan memiliki tujuan target lempar.
138	dx, dy = self.target_pos[0] - self.pos[0], self.target_pos[1] - self.pos[1]	Menghitung jarak horizontal dan vertikal ke posisi target lempar.
139	dist = math.hypot(dx, dy)	Menghitung jarak total tersisa.
140	if dist < self.speed:	Jika sisa jarak lebih kecil dari kecepatan luncur (8px).
141	self.pos, self.reached_target = list(self.target_pos), True	Item langsung tiba di posisi target tujuan dan status reached_target diset True.
142	else:	Jika sisa jarak masih lebih besar dari kecepatan.
143	self.pos[0] += (dx / dist) * self.speed	Menggeser posisi X mendekati target sesuai rasio kecepatan.
144	self.pos[1] += (dy / dist) * self.speed	Menggeser posisi Y mendekati target.
145	self.timer += 1	Menambahkan nilai timer aktif item taktis sebanyak 1 per frame.
146	dur = {"Molotov": 180, "Decoy": 600, "Taser": 10}.get(self.item_type, 100)	Menentukan durasi aktif: Molotov aktif 180 frame (~3 detik), Decoy aktif 600 frame (~10 detik), Taser aktif 10 frame.
147	if self.timer > dur: self.active = False	Jika timer aktif melebihi batas durasi item, set status keaktifan item menjadi False (item hancur/mati).
148	def draw(self, screen, camera):	Metode untuk merender efek visual item taktis ke layar.
149	dp = camera.apply(self.pos)	Menghitung koordinat rendering di layar sesuai dengan pergeseran kamera scroll.
150	if self.item_type == "Molotov" and self.reached_target:	Efek visual Molotov: jika bertipe Molotov dan sudah mendarat di tanah.
151	pygame.draw.circle(screen, (255, 100, 0) if self.timer % 10 < 5 else (255, 200, 0), dp, 50)	Menggambar area api lingkaran di tanah berjejaring 50px yang bergantian warna oranye (255, 100, 0) dan kuning (255, 200, 0) setiap 5 frame untuk efek kedipan api.
152	elif self.item_type == "Decoy":	Efek visual Umpan (Decoy).
153	c = (0, 0, 255) if self.timer % 20 < 10 else (100, 100, 255)	Menggunakan warna biru berkedip setiap 10 frame (biru pekat vs biru terang).

154	<code>pygame.draw.circle(screen, c, dp, 10)</code>	Menggambar lingkaran kecil umpan berjari 10px di tengah.
155	<code>pygame.draw.circle(screen, c, dp, (self.timer % 40) * 2, 1)</code>	Menggambar lingkaran radar pemancar suara yang mengembang keluar secara dinamis (efek gelombang suara penarik zombie).
156	<code>elif self.item_type == "Taser":</code>	Efek visual Taser.
157	<code>pygame.draw.line(screen, (255, 255, 255), dp, (dp[0]+10, dp[1]-10), 2)</code>	Menggambar garis petir kecil berwarna putih (255, 255, 255) setebal 2px dari titik taser ke arah atas-kanan.
158	<i>(Baris Kosong)</i>	Akhir dari berkas kode sumber.

Hubungan dan Alur Penggunaan Entitas

Setiap entitas bekerja sama di bawah naungan game loop utama: * **Pergerakan Pemain:** Pemain diperbarui menggunakan metode `.update()` di game loop Sanctuary atau pertahanan. Pakaian, rambut, dan aksesoris digambar terpisah agar dapat diubah secara dinamis. * **Perilaku AI Zombie:** Ketika diaktifkan di kota pertahanan, `ZombieNPC` memantau `list tactical_items`. Jika pemain melempar Decoy, zombie beralih arah mengejar koordinat umpan alih-alih mengejar pemain. Jika zombie berjalan di atas api Molotov, nyawanya terpangkas cepat dan mengalami status stun. * **Fisika Lemparan Barang:** Ketika pemain menekan klik kanan untuk melempar item taktis (misal Molotov), game memicu pembuatan objek `TacticalItem` baru. Objek tersebut berpindah sumbu X/Y frame demi frame sampai tiba di target klik kursor (`reached_target = True`), lalu memicu efek visual dan damage. * **Sistem Looting:** Ketika zombie musuh dikalahkan, ia dapat meninggalkan item Loot di tanah. Pemain yang mendekati item tersebut (`< 100px`) akan melihat teks petunjuk `get_interaction_prompt()`. Jika menekan tombol Enter, item loot diambil, memicu fungsi internal pemain, dan status item diset `.collected = True` untuk dihapus dari layar.

■ Arsitektur & Alur Aliran Game (City Under Attack)

Dokumen ini menjelaskan struktur kode game Anda secara visual menggunakan flowchart dan diagram. Penjelasan dibagi menjadi 3 level abstraksi sesuai dengan request Anda:

1. **Dari Per-Baris (Line-by-Line) Menjadi Satu Kesatuan yaitu `main()`**
2. **Dari Beberapa Fungsi Menjadi Satu Kelas (Function-to-Class Composition)**
3. **Dari Beberapa Kelas Menjadi Satu "Otak" Game (Class-to-Brain Integration)**

■ 1. Dari Per-Baris (Line-by-Line) Menjadi Satu Kesatuan yaitu `main()`

Diagram di bawah ini memetakan **setiap baris dan blok kode** secara spesifik di dalam file [main.py](#), menunjukkan bagaimana baris-baris kode tersebut terstruktur dan dieksekusi mengalir hingga menjadi satu kesatuan di fungsi `main()`.

```
flowchart TD
    %% Styling
    classDef start_node fill:#ff4444,stroke:#333,stroke-width:2px,color:#fff;
    classDef block_init fill:#ffaa00,stroke:#333,stroke-width:1px,color:#000;
    classDef block_loop fill:#00aa00,stroke:#333,stroke-width:2px,color:#fff;
    classDef branch fill:#cc55ff,stroke:#333,stroke-width:1px,color:#fff;
```

```

classDef render fill:#0088ff,stroke:#333,stroke-width:1px,color:#fff;

%% Entry point
L549["[Baris 549-551] Entry Point:<br>if __name__ == '__main__': main()"]::start_node --> L21["[Baris 21] Panggil

%% Inisialisasi
subgraph Inisialisasi ["Fase Inisialisasi Awal (Baris 22-151)"]
    L21 --> L22["[Baris 22-28] Pygame Setup:<br>pygame.init(), display.set_mode(), clock"]::block_init
    L22 --> L30["[Baris 30-47] Instansiasi Objek:<br>Player, Camera, DialogueBox, Currency, HUD, VFX, SanctuaryLog
    L30 --> L48["[Baris 48-53] Multi-threading CV:<br>Inisialisasi & Start GestureThread"]::block_init
    L48 --> L55["[Baris 55-114] Wardrobe Customization Data:<br>Skin, Outfit, Hair, Hat Options & Indices"]::block
    L55 --> L115["[Baris 115-125] Weapon Upgrade Data:<br>Upgrade costs & status levels"]::block_init
    L115 --> L126["[Baris 126-151] Helper Function & Sync:<br>update_player_wardrobe() & sync awal"]::block_init
end

%% Loop
L126 --> L154["[Baris 154] while True: (Game Loop Utama)"]::block_loop

subgraph Loop_Frame ["Proses per Frame (Baris 155-548)"]
    L154 --> L155["[Baris 155-159] Frame Timing & Sensor Read:<br>dt = clock.tick(60), get_pressed(), gesture_thre

%% Cabang Wardrobe
L155 --> L161{"[Baris 161] Apakah<br>wardrobe_active == True?"}::branch
L161 -- "Ya" --> L162["[Baris 162-194] Handle Key Event Wardrobe<br>(Ganti baju/kulit/topi/rambut)"]::block_lo
L162 --> L195["[Baris 195-308] Draw Background + Overlay Wardrobe Menu"]::render
L195 --> L308["[Baris 308-309] display.flip() & continue"]::render
L308 --> L154

%% Cabang Upgrade
L161 -- "Tidak" --> L312{"[Baris 312] Apakah<br>upgrade_shop_active == True?"}::branch
L312 -- "Ya" --> L313["[Baris 313-332] Handle Key Event Upgrade Shop<br>(Beli Damage/Ammo/Reload/Rate)"]::blo
L313 --> L334["[Baris 334-457] Draw Background + Overlay Upgrade Menu"]::render
L334 --> L457["[Baris 457-458] display.flip() & continue"]::render
L457 --> L154

%% Cabang Minigame
L312 -- "Tidak" --> L463{"[Baris 463] Apakah<br>minigame_manager.in_minigame == True?"}::branch
L463 -- "Ya" --> L464["[Baris 464-487] Handle Input Minigame:<br>Mouse / Smooth Hand Gesture & Recoil/Shoot"]::
L464 --> L488["[Baris 488-490] Update & Draw Active Shooting Game"]::render
L488 --> L490["[Baris 490-491] display.flip() & continue"]::render
L490 --> L154

%% Cabang Default (Eksplorasi)
L463 -- "Tidak" --> L494["[Baris 494-506] Event Handling Mode Petualangan:<br>Lacak tombol QUIT & ENTER untuk o
L494 --> L508["[Baris 508-513] Input Mapping (Keyboard + CV):<br>Mapping Gestur ATAS/BAWAH/KIRI/KANAN/PISTOL"]
L508 --> L516{"[Baris 516] Apakah<br>dialogue.active == False?"}::branch

%% Update Logic Eksplorasi
L516 -- "Ya (Dialog Tidak Aktif)" --> L517["[Baris 517-518] Update Fisika Pemain & Kamera:<br>player.update() &
L517 --> L521["[Baris 521-531] Update Misi/NPC SanctuaryLogic:<br>Kembalikan sinyal start shooting/koin/wardrob
L516 -- "Tidak (Sedang Dialog)" --> L533["Lewati update pemain, fokus ke dialog"]::block_loop

%% Render Akhir Eksplorasi
L521 --> L533["[Baris 533-535] Render Map & Karakter:<br>draw_ground(), draw_entities(), player.draw()"]::ren
L533 --> L538["[Baris 538-540] Render Overlay UI:<br>VFX flash, dialogue box, HUD sender"]::render
L538 --> L543["[Baris 543-545] Render Status Text / Objektif Misi"]::render
L543 --> L547["[Baris 547] pygame.display.flip()"]::render
L547 --> L154
end

```

■ 2. Dari Beberapa Fungsi Menjadi Satu Kelas

Bagian ini menunjukkan bagaimana **Data/State** dan **Fungsi-Fungsi** yang tersebar dikemas menjadi satu kesatuan di dalam kelas [Player](#). Semua fungsi di dalam kelas berinteraksi menggunakan parameter internal (self).

```
classDiagram
    class Player {
        +list pos [x, y]
        +Spritesheet sheet (Base Skin)
        +Spritesheet clothing_sheet
        +Spritesheet hair_sheet
        +Spritesheet hat_sheet
        +string direction
        +string state
        +int current_col
        +int health
        +float speed_multiplier
        +int adrenaline_timer

        +__init__(x, y)
        +update(keys, collision_mask, map_size)
        +draw(screen, camera)
    }

    note for Player "Fungsi __init__:<br>Inisialisasi semua variabel status pemain."
    note for Player "Fungsi update:<br>Memproses input tombol/gestur, mengubah posisi, dan animasi."
    note for Player "Fungsi draw:<br>Menggambar skin, baju, rambut, dan topi secara bertumpuk (layering) di layar."
```

Penjelasan Hubungan Fungsi & Data di dalam Class:

1. **self (Jembatan Data):** Variabel seperti `self.pos`, `self.direction`, dan `self.current_col` adalah data bersama.
2. **update() Mengubah Data:** Fungsi ini membaca input keyboard/gestur dan **mengubah** data posisi (`self.pos`) serta indeks animasi (`self.current_col`).
3. **draw() Membaca Data:** Fungsi ini dipanggil setelah `update()` untuk membaca data terkini (`self.pos`, `self.direction`, `self.current_col`) dan **merendernya** ke layar menggunakan Pygame.

■ 3. Dari Beberapa Kelas Menjadi Satu "Otak" Game (Central Brain)

Inilah arsitektur tertinggi game Anda. **Otak** dari game Anda adalah **Game Loop & Orchestrator** yang berada di dalam [main.py](#) pada fungsi `main()`. Fungsi ini mengatur aliran data antar kelas yang berbeda agar saling bekerja sama.

```
graph TD
    classDef brain fill:#ff3399,stroke:#333,stroke-width:3px,color:#fff;
    classDef core fill:#33ccff,stroke:#333,stroke-width:1px,color:#000;
    classDef ui fill:#cc99ff,stroke:#333,stroke-width:1px,color:#000;
    classDef sensor fill:#33cc66,stroke:#333,stroke-width:1px,color:#000;

    %% Central Brain
    Brain["■ OTAK GAME<br>(main.py - Main Loop)"]:::brain

    %% Subsystems / Classes
    CV["■ GestureThread (Computer Vision)"]:::sensor
```

```

Logic["■ SanctuaryLogic (Sistem Map/NPC)"]:::core
Pl["■ Player (Pemain)"]:::core
Cam["■ Camera (Viewport Kamera)"]:::core
Minigame["■ MiniGameManager (Shooting Mode)"]:::core
HUD_Cl["■ HUD (Status Tampilan)"]:::ui
Diag["■ DialogueBox (Sistem Narasi)"]:::ui
Cur["■ CurrencyManager (Ekonomi Game)"]:::core
VFX["■ VisualEffects (VFX/Screen Shake)"]:::core

%% Connections (Data Flow)
CV -- "1. Deteksi Gestur Tangan<br>(Gesture Terkini & Posisi)" --> Brain
Brain -- "2. Input Proxy (Keys/Gestures)" --> Pl
Pl -- "3. Posisi Terkini" --> Cam
Brain -- "4. Kirim Sinyal Interaksi<br>& Status Pemain" --> Logic
Logic -- "5. Minta Tampilkan Teks" --> Diag
Logic -- "6. Trigger Event Pemicu<br>(Buka Wardrobe/Upgrade)" --> Brain
Logic -- "7. Switch State 'START_SHOOTING'" --> Brain
Brain -- "8. Aktifkan Shooting Mode" --> Minigame
Minigame -- "9. Beri Koin Reward" --> Cur
Cur -- "10. Data Uang Terkini" --> HUD_Cl
Brain -- "11. Gambar Semua Frame" --> VFX
VFX -- "12. Render Final ke Layar" --> Brain

```

Deskripsi Aliran Kerja "Otak" Game:

- **Pengumpul Sensor ([GestureThread](#)):** Berjalan di thread latar belakang untuk memproses input kamera lewat MediaPipe secara real-time. Hasilnya dikirim ke **Otak Game** berupa nama gestur (seperti "ATAS", "PISTOL") dan koordinat tangan.
- **Koordinator Input: Otak Game** membungkus input keyboard dan gestur kamera menjadi satu proxy input (InputProxy) lalu meneruskannya ke kelas [Player](#).
- **Pengelola State & Event ([SanctuaryLogic](#)):** Menghitung interaksi pemain dengan dunia (seperti api unggun, kasur medis, NPC Kapten Jaka/Penatua Aris, helipad supply drop). Jika terjadi event khusus, ia mengirimkan sinyal string ke **Otak Game** (contoh: "START_SHOOTING", "OPEN_WARDROBE", "AWARD_150_BRONZE").
- **Peralihan Mode ([MiniGameManager](#)):** Ketika menerima sinyal "START_SHOOTING", **Otak Game** menghentikan rendering eksplorasi dan mengalihkan kendali penuh ke mode pertempuran FPS melawan zombie.

Penjelasan Kode: `gestures.py`

Dokumen ini berisi penjelasan mendetail baris demi baris mengenai file `/home/abuyyy/PemogramanLanjut/comvis/gestures.py` dalam bahasa Indonesia untuk membantu persiapan demo program.

Deskripsi & Tujuan File

File `gestures.py` bertanggung jawab atas seluruh proses visi komputer (Computer Vision) menggunakan MediaPipe dan OpenCV. Tugas utamanya adalah: 1. Menangkap frame video dari kamera secara real-time di thread terpisah (`GestureThread`). 2. Mendeteksi landmark tangan dan membagi logika deteksi gestur menjadi dua mode: - **Mode Eksplorasi Sanctuary:** Menggunakan model Machine Learning (`gesture_model.pkl`) untuk mendeteksi ATAS, BAWAH, KIRI, KANAN, dan AMBIL. - **Mode Minigame:** Menggunakan perhitungan matematika rasio jari (bebas rotasi & skala) untuk mendeteksi PISTOL (menembak), AIM (membidik saja), dan FIST (mengisi peluru). 3. Menerapkan filter `OneEuroFilter` untuk menstabilkan pergerakan cursor agar sangat halus.

Daftar Import

Library	Kegunaan
cv2	OpenCV, digunakan untuk membaca kamera, membalik gambar, konversi warna (BGR ke RGB), serta mengubah ukuran frame.
mediapipe	Framework ML Google untuk mendeteksi 21 titik koordinat (landmark) tangan secara real-time.
pickle	Digunakan untuk memuat model Machine Learning yang telah dilatih (<code>gesture_model.pkl</code>).
os	Berinteraksi dengan sistem operasi, memuat file path model.
numpy	Penanganan komputasi matriks/array (digunakan oleh pemrosesan gambar).
threading	Menjalankan kamera pada Thread terpisah agar tidak mengganggu thread utama game.
warnings	Memblokir peringatan abaikan dari scikit-learn/mediapipe agar konsol tetap bersih.
pandas	Membuat DataFrame untuk input data ke model prediksi ML.
math	Operasi matematika dasar seperti menghitung jarak Euclidean (<code>math.hypot</code>).
collections	Menggunakan deque sebagai buffer geser untuk meredam kedipan (<i>flicker</i>) gestur.
time	Menghitung interval waktu untuk filter OneEuroFilter dan jeda tembakan.

Penjelasan Baris Demi Baris

Baris	Kode Sumber	Penjelasan
1	<code>import cv2</code>	Mengimpor OpenCV untuk operasi video dan gambar.
2	<code>import mediapipe as mp</code>	Mengimpor MediaPipe untuk pelacakan tangan.
3	<code>import pickle</code>	Mengimpor pickle untuk deserialisasi model ML.
4	<code>import os</code>	Mengimpor os untuk penanganan path file.
5	<code>import numpy as np</code>	Mengimpor numpy untuk penanganan data array.
6	<code>import threading</code>	Mengimpor threading untuk memisahkan proses kamera.
7	<code>import warnings</code>	Mengimpor warnings untuk mengabaikan UserWarning.
8	<code>import pandas as pd</code>	Mengimpor pandas untuk konversi data input model ML.
9	<code>import math</code>	Mengimpor math untuk kalkulasi koordinat dan jarak.
10	<code>import collections</code>	Mengimpor collections untuk antrean geser (deque).
11	<code>import time</code>	Mengimpor time untuk pengukuran waktu filter.
12	<code>warnings.filterwarnings("ignore", category=UserWarning)</code>	Menonaktifkan peringatan UserWarning agar output konsol tetap bersih.
13	<code>os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'</code>	Membatasi log TensorFlow agar hanya menampilkan error fatal.
14	<code>class GestureRecognizerML:</code>	Mendefinisikan kelas deteksi gestur menggunakan Machine Learning & Heuristik.

15	def __init__(self):	Konstruktor kelas deteksi gestur.
16	self.mp_hands = mp.solutions.hands	Mengambil referensi modul Hands dari MediaPipe.
17	self.hands = self.mp_hands.Hands(...)	Konfigurasi detektor tangan MediaPipe (max 1 tangan, konfidensi deteksi 70%).
23	self.mp_draw = mp.solutions.drawing_utils	Mengambil utilitas penggambaran garis landmark tangan dari MediaPipe.
24	base_dir = os.path.dirname(os.path.abspath(__file__))	Mencari path direktori dari file gestures.py.
25	self.model_path = os.path.join(base_dir, 'gesture_model.pkl')	Menentukan lokasi file model ML (gesture_model.pkl).
26	self.model = None	Menginisialisasi model dengan nilai None.
27	if os.path.exists(self.model_path):	Mengecek apakah file model ML ada di penyimpanan disk.
28	with open(self.model_path, 'rb') as f:	Membuka file model biner tersebut untuk dibaca.
29	self.model = pickle.load(f)	Memuat file model biner ke atribut self.model.
30	print("Model Machine Learning berhasil dimuat.")	Cetak status sukses memuat model ke terminal.
31	else:	Jika file model tidak ditemukan.
32	print("Peringatan: Model belum dilatih. Gunakan train_model.py!")	Cetak status peringatan ke terminal.
33	def recognize(self, hand_landmarks, in_minigame=False):	Metode utama untuk mengenali gestur berdasarkan status permainan.
34	lm = hand_landmarks.landmark	Menyimpan referensi 21 titik koordinat tangan.
35	if in_minigame:	LOGIKA MINIGAME:
36	has_ml_minigame = False	Inisialisasi status apakah model ML sudah memiliki kelas minigame.
37	if self.model is not None and hasattr(self.model, 'classes_'):	Memeriksa apakah model dimuat dan memiliki daftar kelas terlatih.
38	has_ml_minigame = any(c in self.model.classes_ for c in ["PISTOL", "AIM", "FIST"])	Memeriksa keberadaan kelas PISTOL, AIM, dan FIST di dalam model ML.
40	if has_ml_minigame:	Jika model ML mendukung gestur minigame, gunakan ML untuk prediksi.
41	# ML Prediction logic ...	Mengekstrak landmark, membuat DataFrame, memprediksi label dengan Random Forest.
48	if pred_label in ["PISTOL", "AIM", "FIST"]:	Mengembalikan label minigame jika terdeteksi.
50	return pred_label	
51	else:	LOGIKA FALLBACK (Heuristik Matematika): Terpilih jika model belum dilatih.
52	hand_scale = math.hypot(...)	Menghitung skala ukuran tangan.
62	if is_index_open and is_middle_folded and is_ring_folded and is_pinky_folded:	Aturan deteksi: telunjuk lurus dan jari lain tertekuk.
63	return "PISTOL" if is_thumb_loose else "AIM"	Kembalikan PISTOL jika jempol terbuka, AIM jika jempol ditekuk.

69	<pre> if is_index_folded and is_middle_folded_strict and is_ring_folded_strict and is_pinky_folded_strict: </pre>	Aturan deteksi: genggaman tangan (fist).
70	<pre> return "FIST" </pre>	Mengembalikan status FIST (Reload).
71	<pre> return "None" </pre>	Default kembali ke None.
72	<pre> else: </pre>	LOGIKA EKSPLORASI
73	<pre> pred_label = "None" </pre>	SANCTUARY (Selalu ML):
74	<pre> if self.model is not None: </pre>	Inialisasi default label.
77-80	<pre> for lm_node in hand_landmarks.landmark: data.append(...) </pre>	Jika model ML aktif. Mengumpulkan koordinat X, Y, Z relatif terhadap pergelangan tangan (wrist).
81	<pre> columns = ... df_input = </pre>	Membuat nama kolom dataset.
82	<pre> pd.DataFrame([data], columns=columns) </pre>	Mengonversi ke DataFrame Pandas.
83	<pre> prediction = self.model.predict(df_input) </pre>	Memprediksi gestur dengan RandomForest.
84	<pre> pred_label = prediction[0] </pre>	Mengambil label hasil prediksi.
85	<pre> if pred_label in ["ATAS", "BAWAH", "KIRI", "KANAN", "AMBIL"]:</pre>	Memastikan hasil prediksi adalah tombol navigasi yang valid.
86	<pre> return pred_label </pre>	Mengembalikan hasil label navigasi.
87	<pre> return "None" </pre>	Mengembalikan None jika tidak terdeteksi.
78	<pre> class OneEuroFilter: </pre>	Mendefinisikan kelas filter One Euro untuk melembutkan fluktuasi koordinat.
102	<pre> class GestureThread(threading.Thread): </pre>	Kelas Thread untuk menangkap frame kamera dan melakukan deteksi tangan secara terpisah.
103	<pre> def __init__(self, camera_path): </pre>	Inialisasi thread dengan path kamera (index 0).
119	<pre> self.in_minigame = False </pre>	Status apakah pemain sedang berada di dalam minigame (default False).
123	<pre> @property def current_gesture(self): </pre>	Properti thread untuk mengambil gestur aktif saat ini secara aman (<i>thread-safe</i>).
126	<pre> @property def hand_pos(self): </pre>	Properti thread untuk mengambil koordinat X dan Y ujung telunjuk secara aman.
129	<pre> @property def recoil_active(self): </pre>	Mendeteksi apakah sentakan/tembakan baru saja terpicu, lalu langsung mereset status recoil tersebut ke False.
139	<pre> def run(self): </pre>	Fungsi utama loop kamera yang berjalan di latar belakang.
141	<pre> cap = cv2.VideoCapture(self.camera_path) </pre>	Membuka akses kamera menggunakan OpenCV.
146	<pre> while self.running: </pre>	Selama status thread berjalan.
147	<pre> ret, frame = cap.read() </pre>	Membaca frame terbaru dari kamera.
150	<pre> flipped_frame = cv2.flip(frame, 1) rgb_frame = </pre>	Membalik gambar secara horizontal agar bertindak seperti cermin.
151	<pre> cv2.cvtColor(flipped_frame, cv2.COLOR_BGR2RGB) </pre>	Mengonversi warna BGR (OpenCV) ke RGB (MediaPipe).
152	<pre> results = self.recognizer.hands.process(rgb_frame) </pre>	Memproses gambar RGB untuk mencari titik koordinat tangan.

154	if	Jika ada tangan terdeteksi di dalam frame.
	results.multi_hand_landmarks:	
	gesture =	
158	self.recognizer.recognize(hand_landmarks,	Mengenali gestur tangan frame saat ini.
	self.in_minigame)	
159	self._gesture_buffer.append(gesture)	Memasukkan hasil ke dalam buffer deque berkapasitas 5.
	smoothed_gesture =	
160	max(set(self._gesture_buffer),	Mengambil gestur terpopuler di buffer untuk meredam noise (majority vote).
	key=self._gesture_buffer.count)	
	is_shooting_g =	
161	smoothed_gesture ==	Cek apakah gestur saat ini adalah "PISTOL" (Tembak).
	"PISTOL"	
	was_shooting_g =	
162	self._prev_gesture ==	Cek apakah gestur sebelumnya adalah "PISTOL".
	"PISTOL"	
	transition_to_shoot =	
163	is_shooting_g and not	Terpicu jika baru saja berubah dari tidak menembak ke "PISTOL" (tembakan pertama).
	was_shooting_g	
	self._prev_gesture =	Menyimpan gestur saat ini untuk pengecekan frame berikutnya.
164	smoothed_gesture	
	tip =	Titik landmark 8 (ujung jari telunjuk).
165	hand_landmarks.landmark[8]	
	raw_v_y = self._last_y -	Menghitung kecepatan gerakan Y jari telunjuk.
166	tip.y	
	self._velocity_y = (alpha	Menerapkan eksponensial moving average pada kecepatan gerakan Y.
167	* raw_v_y) + (1.0 - alpha)	
	* self._velocity_y	
168	with self._lock:	Mengamankan variabel bersama menggunakan lock mutex.
	self._current_gesture =	Menyimpan gestur ke atribut thread.
169	smoothed_gesture	
170	t_now = time.time()	Mengambil waktu saat ini.
	self._hand_pos[0] =	Menyaring koordinat X ujung telunjuk menggunakan
171	self.filter_x(t_now,	OneEuroFilter.
	tip.x)	
	self._hand_pos[1] =	Menyaring koordinat Y ujung telunjuk menggunakan
172	self.filter_y(t_now,	OneEuroFilter.
	tip.y)	
	jerk_fired =	Sentakan terpicu jika gestur menembak aktif dan ada gerakan kejutan telunjuk ke atas > 0.025.
173	(is_shooting_g and	
	self._velocity_y > 0.025)	Jika tembakan pertama terpicu ATAU ada sentakan tangan ke atas.
174	if (transition_to_shoot or	
	jerk_fired):	
	if time.time() -	
175	self._last_shot_time >	Mencegah tembakan bertumpuk terlalu cepat (jeda minimal 250ms).
	0.25:	
176	self._recoil_triggered =	Aktifkan sinyal tembakan/recoil untuk dibaca game.
	True	
	self._last_shot_time =	Perbarui pencatatan waktu tembakan terakhir.
177	time.time()	
	self._last_y = tip.y	Simpan koordinat Y telunjuk saat ini untuk frame berikutnya.
178		
179	else:	Jika tidak ada tangan terdeteksi.
	with self._lock:	
180	self._current_gesture =	Atur gestur ke "None".
	"None"	
181	import pygame	Mengimpor pygame secara lokal untuk membuat surface gambar.
	small =	Memperkecil resolusi frame video menjadi 160x120 piksel untuk overlay layar.
182	cv2.resize(flipped_frame,	
	(160, 120))	

183	<code>rgb_small = cv2.cvtColor(small, cv2.COLOR_BGR2RGB) surf = pygame.surfarray.make_surface(rgb_small.swapaxes(0, 1)) with self._lock: self.latest_frame = surf cap.release() self.running = False</code>	Mengonversi format warna frame kecil ke RGB. Membuat surface pygame dari numpy array gambar kecil tersebut. Mengunci thread. Menyimpan surface frame video terbaru untuk digambar di layar game. Melepas perangkat kamera setelah loop berhenti. Mengubah status bendera loop menjadi mati untuk menghentikan thread kamera.
# Penjelasan Kode: hud.py		

Dokumen ini berisi penjelasan mendetail baris demi baris mengenai file `/home/abuyyy/PemogramanLanjut/src/ui/hud.py` dalam bahasa Indonesia untuk membantu persiapan demo program.

Deskripsi & Tujuan File

File `hud.py` mendefinisikan kelas HUD (Heads-Up Display). Kelas ini berfungsi untuk menggambar antarmuka informasi status di layar game (seperti di pojok kiri atas). Informasi yang ditampilkan meliputi: 1. **Mata Uang (Currency)**: Menampilkan koin emas (Gold), perak (Silver), dan perunggu (Bronze) dengan efek transisi angka yang mulus (animasi *lerp*). 2. **Status Bar**: Menampilkan bar nyawa (Health) berwarna merah dan sisa daya baterai (Battery) berwarna biru.

Daftar Import

Library	Kegunaan
<code>import pygame</code>	Library utama untuk membuat game 2D di Python. Digunakan di sini untuk menggambar bentuk geometris (lingkaran, persegi), memanipulasi permukaan grafis (<i>surface</i>), serta memuat dan merender font/teks.
<code>import time</code>	Modul bawaan Python untuk memanipulasi waktu. Di dalam file ini, modul diimpor tetapi belum digunakan secara aktif.

Penjelasan Baris Demi Baris

Berikut adalah tabel analisis dari setiap baris kode di dalam `hud.py`:

Baris	Kode Sumber	Penjelasan
1	<code>import pygame</code>	Mengimpor modul pygame untuk menangani rendering grafik dan teks.
2	<code>import time</code>	Mengimpor modul time bawaan Python.
3	<code>class HUD:</code>	Mendefinisikan kelas HUD yang bertanggung jawab merender antarmuka status pemain.
4	<code>def __init__(self, currency):</code>	Konstruktor kelas HUD yang menerima parameter currency (objek pelacak mata uang pemain).
5	<code>self.currency = currency</code>	Menyimpan referensi objek currency ke dalam atribut objek <code>self.currency</code> .

6	self.font_main = pygame.font.SysFont("Arial", 18, bold=True)	Inisialisasi font Arial ukuran 18 tebal (<i>bold</i>) untuk nilai angka mata uang.
7	self.font_sub = pygame.font.SysFont("Arial", 12)	Inisialisasi font Arial ukuran 12 untuk label teks kecil di atas status bar.
8	self.display_gold, self.display_silver, self.display_bronze = currency.gold, currency.silver, currency.bronze	Menginisialisasi nilai mata uang tampilan awal berdasarkan nilai aktual untuk keperluan animasi smooth.
9	self.CLR_BG, self.CLR_ACCENT, self.CLR_WHITE = (20, 20, 20, 180), (255, 215, 0), (240, 240, 240)	Warna Palette: Latar panel abu-abu gelap transparan, aksen emas koin, dan warna teks putih abu-abu terang.
10	self.CLR_HEALTH, self.CLR_STAMINA, self.CLR_BATTERY = (231, 76, 60), (46, 204, 113), (52, 152, 219)	Warna Palette: Status bar kesehatan merah, stamina hijau, dan baterai biru.
11	def _draw_bar(self, screen, x, y, width, label, value, max_value, color):	Metode internal untuk menggambar status bar (seperti health/battery).
12	pygame.draw.rect(screen, (40, 40, 40), (x, y, width, 8), border_radius=4)	Menggambar bar latar belakang berwarna abu-abu gelap dengan tebal 8 piksel dan sudut melengkung.
13	fill_w = int((value / max_value) * width)	Menghitung lebar isi bar secara proporsional sesuai rasio nilai saat ini terhadap nilai maksimum.
14	if fill_w > 0:	Memastikan lebar isi bar lebih dari 0 sebelum melakukan penggambaran untuk menghindari bug visual.
15	pygame.draw.rect(screen, color, (x, y, fill_w, 8), border_radius=4)	Menggambar isi bar sesuai warna parameter di atas bar latar belakang dengan sudut melengkung.
16	lbl_surf = self.font_sub.render(label, True, (150, 150, 150))	Membuat permukaan teks (<i>surface</i>) untuk label dengan font berukuran kecil dan warna abu-abu.
17	screen.blit(lbl_surf, (x, y - 15))	Menempelkan (<i>blit</i>) permukaan label di layar pada posisi 15 piksel di atas bar.
18	def draw(self, screen, player=None, battery_level=100):	Metode utama untuk menggambar seluruh tampilan HUD ke layar. Menerima objek player dan tingkat baterai.
19	panel_rect = pygame.Rect(20, 20, 260, 100)	Membuat objek pygame.Rect untuk menentukan ukuran dan posisi panel (posisi x=20, y=20, lebar=260, tinggi=100).
20	bg_surf = pygame.Surface((panel_rect.width, panel_rect.height), pygame.SRCALPHA)	Membuat <i>surface</i> baru transparan dengan format SRCALPHA untuk area panel.
21	pygame.draw.rect(bg_surf, self.CLR_BG, bg_surf.get_rect(), border_radius=10)	Menggambar persegi panjang sudut melengkung berwarna transparan gelap di atas <i>surface</i> panel tersebut.

22	<pre> screen.blit(bg_surf, (panel_rect.x, panel_rect.y)) self.display_gold += (self.currency.gold - self.display_gold) * 0.1; self.display_silver += </pre>	Menempelkan panel latar belakang transparan ke layar utama game.
23	<pre> (self.currency.silver - self.display_silver) * 0.1; self.display_bronze += (self.currency.bronze - self.display_bronze) * 0.1 </pre>	Mendekatkan nilai koin emas, perak, dan perunggu tampilan ke nilai aktual sebanyak 10% di setiap frame secara smooth.
24	<pre> currencies = [</pre>	Memulai pendefinisian list currencies yang menampung data rendering untuk setiap koin.
25	<pre> ("G", int(self.display_gold), (255, 215, 0)), </pre>	Data koin emas: label koin "G", nilai koin bulat (diambil dari nilai animasi), dan warna lingkaran koin (Emas).
26	<pre> ("S", int(self.display_silver), (192, 192, 192)), </pre>	Data koin perak: label koin "S", nilai koin bulat, dan warna lingkaran koin (Perak).
27	<pre> ("B", int(self.display_bronze), (205, 127, 50)) </pre>	Data koin perunggu: label koin "B", nilai koin bulat, dan warna lingkaran koin (Perunggu).
28	<pre>] </pre>	Penutup list dari data koin.
29	<pre> start_x = 40 </pre>	Menentukan titik awal koordinat X untuk menggambar koin pertama.
30	<pre> for icon, val, color in currencies: </pre>	Melakukan perulangan (<i>looping</i>) untuk menggambar masing-masing koin dari list currencies.
31	<pre> pygame.draw.circle(screen, color, (start_x, 45), 8) </pre>	Menggambar lingkaran kecil dengan warna koin pada posisi y=45 dengan jari-jari 8 piksel.
32	<pre> val_surf = self.font_main.render(str(val), True, self.CLR_WHITE) </pre>	Merender teks angka nilai koin (yang sedang teranimasi) menjadi objek <i>surface</i> .
33	<pre> screen.blit(val_surf, (start_x + 15, 35)) </pre>	Menempelkan teks nilai koin tersebut di sebelah kanan lingkaran koin (jarak offset 15 piksel).
34	<pre> start_x += 75 </pre>	Menggeser posisi X ke kanan sebesar 75 piksel untuk koin berikutnya agar tidak bertumpukan.
35	<pre> hp = player.health if player else 100 </pre>	Mengambil nilai kesehatan pemain. Jika objek <i>player</i> bernilai <i>None</i> , nilai kesehatannya otomatis diatur ke 100.
36	<pre> stamina = player.stamina if hasattr(player, 'stamina') else 100 </pre>	Mengambil nilai stamina jika objek <i>player</i> memiliki atribut <i>stamina</i> , jika tidak diatur ke 100.
37	<pre> bar_x = 40 </pre>	Mengatur koordinat horizontal X awal untuk bar status.
38	<pre> self._draw_bar(screen, bar_x, 80, 200, "HEALTH", hp, 100, self.CLR_HEALTH) self._draw_bar(screen, </pre>	Menggambar bar kesehatan berwarna merah pada posisi y=80 dengan lebar maksimal 200 piksel.
39	<pre> bar_x, 105, 200, "BATTERY", battery_level, 100, self.CLR_BATTERY) </pre>	Menggambar bar baterai berwarna biru pada posisi y=105 dengan lebar maksimal 200 piksel.
40	<pre> (baris kosong) </pre>	Pemisah visual/akhir dari file kode program.
# Penjelasan Kode: main.py		

Deskripsi & Tujuan File

File `main.py` adalah pintu masuk utama (*entry point*) peluncuran game. File ini bertanggung jawab untuk: 1. **Inisialisasi Game**: Membuka jendela layar Pygame, memuat font, mengeset setelan dasar, dan memuat aset kustomisasi pakaian pemain (wardrobe) serta data stasiun peningkatan senjata (upgrade shop). 2. **Koordinasi Thread Kamera**: Meluncurkan thread kamera visi komputer (`GestureThread`) secara paralel agar tidak menghambat game loop utama. 3. **Menu Kustomisasi Wardrobe & Upgrade Shop**: Menangani navigasi antarmuka visual overlay menu kustomisasi sprite pemain dan toko statistik senjata (damage, amunisi, reload, firerate). 4. **Logika Input Proxy & Gestur**: Mengubah gestur navigasi tangan (ATAS, BAWAH, KIRI, KANAN, AMBIL) menjadi masukan keyboard simulasi (`InputProxy`), serta menskalakan koordinat tangan webcam menjadi koordinat crosshair di layar game dengan interpolasi linear (`lerp`) agar halus. 5. **Game Loop**: Menangani polling event, pembaruan frame, sinkronisasi status thread kamera, dan render berlapis.

Penjelasan Baris Demi Baris

Berikut adalah tabel penjelasan baris demi baris dari kode sumber `main.py`:

Baris	Kode Sumber	Penjelasan
1	<code>import pygame</code>	Mengimpor pustaka Pygame untuk rendering grafis, event, font, dan mixer audio.
2	<code>import sys</code>	Mengimpor modul <code>sys</code> untuk penutupan bersih aplikasi dan python.
3	<code>from src.entities.entities import Player</code>	Mengimpor kelas <code>Player</code> dari modul entitas.
4	<code>from src.ui.dialogue import DialogueBox</code>	Mengimpor kelas <code>DialogueBox</code> untuk memproses kotak teks dialog percakapan.
5	<code>from src.ui.hud import HUD</code>	Mengimpor kelas HUD untuk status bar nyawa, baterai, dan koin.
6	<code>from src.core.engine import CurrencyManager, Camera, VisualEffects, MiniGameManager</code>	Mengimpor manajer mata uang, kamera gulir, efek visual gelap/shake, dan manager daur hidup minigame dari core engine.
7	<code>from src.core.sanctuary_logic import SanctuaryLogic</code>	Mengimpor logika dunia aman Sanctuary.
8	<code>from comvis.gestures import GestureThread</code>	Mengimpor thread webcam visi komputer untuk pelacakan gestur tangan.
9	<code>class InputProxy:</code>	Deklarasi kelas <code>InputProxy</code> untuk memodifikasi masukan status tombol keyboard game.
10	<code>def __init__(self, keys_pressed):</code>	Konstruktur kelas input proxy yang menerima daftar status tombol keyboard fisik saat ini.
11	<code>self.keys = keys_pressed</code>	Menyimpan status tombol keyboard fisik.
12	<code>self.overrides = {}</code>	Inisialisasi kamus kosong untuk menampung tombol keyboard buatan (overrides) dari gestur tangan.
13	<code>def __getitem__(self, key):</code>	Metode pemetaan item (<code>__getitem__</code>) agar objek input proxy dapat diakses menggunakan indeks bracket <code>proxy[key]</code> .
14	<code>return self.overrides.get(key, self.keys[key])</code>	Mengembalikan status buatan dari gestur jika terdaftar, jika tidak kembalikan status keyboard fisik.

15	<code>def main():</code>	Deklarasi fungsi utama pengekseskusi program game.
16	<code>pygame.init()</code>	Menginisialisasi seluruh subsistem Pygame.
17	<code>WIDTH, HEIGHT = 1280, 720</code>	Menetapkan resolusi layar game: lebar 1280px dan tinggi 720px (720p).
18	<code>screen = pygame.display.set_mode((WIDTH, HEIGHT))</code>	Membuat jendela tampilan layar Pygame.
19	<code>pygame.display.set_caption("City Under Attack - Sanctuary Defense")</code>	Mengeset judul jendela permainan di layar desktop.
20	<code>clock = pygame.time.Clock()</code>	Membuat objek pewaktu Clock untuk membatasi framerate game.
21	<code>font = pygame.font.SysFont("monospace", 14)</code>	Inisialisasi font default monospace berukuran 14px untuk UI bawah dan status.
22	<code>weapon_upgrades = { "damage_level": 0, "ammo_level": 0, "reload_level": 0, "firerate_level": 0 }</code>	Kamus level peningkatan statistik senjata awal (seluruh level mulai dari 0 s.d 3).
23	<code>selected_weapon = "SCAR"</code>	Senjata default yang terpilih di dunia eksplorasi: "SCAR".
24	<code>player = Player(500, 450)</code>	Membuat instansiasi karakter pemain diposisikan awal di (500, 450).
25	<code>camera = Camera(WIDTH, HEIGHT, 2560, 1440)</code>	Membuat kamera dengan ukuran layar 1280x720 untuk peta berdimensi 2560x1440.
26	<code>dialogue = DialogueBox()</code>	Membuat kotak dialog percakapan.
27	<code>currency = CurrencyManager()</code>	Membuat manajer keuangan pemain.
28	<code>hud = HUD(currency)</code>	Membuat antarmuka status HUD yang terhubung dengan data mata uang koin.
29	<code>effects = VisualEffects(WIDTH, HEIGHT)</code>	Membuat pengelola visual efek getaran dan gelap malam.
30	<code>game_logic = SanctuaryLogic(dialogue)</code>	Membuat instansiasi logika Sanctuary dengan menyertakan dialogue box.
31	<code>dialogue.box_rect = pygame.Rect(100, 500, 1080, 150)</code>	Mengubah ukuran kotak dialog agar lebih lebar dan berada di bagian tengah bawah layar game.
32	<code>gesture_thread = GestureThread(0)</code>	Membuat thread kamera webcam terpisah menggunakan webcam berindeks 0.
33	<code>gesture_thread.start()</code>	Menjalankan thread kamera pelacak gestur di latar belakang secara paralel.
34	<code>class EngineProxy:</code>	Deklarasi kelas perantara EngineProxy untuk meneruskan status game ke minigame manager.
35	<code>def __init__(self, s, c, cur, upg, gt, sw):</code>	Konstruktor kelas proxy menerima screen, clock, currency, upgrades, thread, dan senjata terpilih.
36	<code>self.screen = s</code>	Menyimpan referensi screen utama.
37	<code>self.clock = c</code>	Menyimpan referensi clock utama.
38	<code>self.currency = cur</code>	Menyimpan referensi currency manager.
39	<code>self.weapon_upgrades = upg</code>	Menyimpan kamus upgrade senjata.

40	self.gesture_thread = gt	Menyimpan referensi thread gestur.
41	self.selected_weapon = sw	Menyimpan nama senjata aktif.
	engine_proxy =	
42	EngineProxy(screen, clock, currency, weapon_upgrades, gesture_thread, "SCAR")	Instansiasi objek proxy engine dengan data awal.
43	minigame_manager =	Membuat pengelola minigame yang terhubung dengan proxy.
	MiniGameManager(engine_proxy)	
44	smoothed_hx = WIDTH // 2	Inisialisasi koordinat horizontal bidikan halus di tengah layar.
45	smoothed_hy = HEIGHT // 2	Inisialisasi koordinat vertikal bidikan halus di tengah layar.
46-50	SKIN_OPTIONS = [...], SKIN_NAMES = [...]	Membuat daftar pilihan 11 path spritesheet warna kulit pemain dan penamaannya.
51-65	OUTFIT_OPTIONS = [...]	Membuat daftar pilihan 13 baju dan celana pemain (dari pakaian forester hingga celana petani).
66-80	OUTFIT_NAMES = [...]	Nama tampilan untuk pilihan pakaian pemain di UI menu wardrobe.
81-88	HAIR_OPTIONS = [...], HAIR_NAMES = [...]	Loop otomatis untuk memuat 28 pilihan file spritesheet gaya rambut pemain (Bob cut & Dapper).
89-96	HAT_OPTIONS = [...], HAT_NAMES = [...]	Loop otomatis memuat 10 pilihan file spritesheet topi pemain (topi jerami petani & topi sihir pointy).
97	wardrobe_active = False	Status keaktifan menu wardrobe kustomisasi pakaian pemain (default False).
98	wardrobe_category = 0	Indeks kategori menu wardrobe yang sedang disorot (0: Kulit, 1: Pakaian, 2: Rambut, 3: Topi).
99	wardrobe_indices = [0, 3, 23, 0]	Indeks item aktif yang dipilih pemain untuk masing-masing kategori kustomisasi.
100	upgrade_shop_active = False	Status keaktifan menu Upgrade Senjata (default False).
101	upgrade_shop_category = 0	Indeks kategori stasiun upgrade yang disorot (0: Damage, 1: Ammo, 2: Reload, 3: Firerate).
102	UPGRADE_COSTS = [100, 250, 500]	Tarif biaya peningkatan: Lv 1 ke 2 butuh 100 Bronze, Lv 2 ke 3 butuh 250 Bronze, Lv 3 ke Max butuh 500 Bronze.
103	(Komentar)	Penjelas.
104	def	Fungsi lokal untuk memuat ulang
	update_player_wardrobe():	lembar sprite visual pemain berdasarkan wardrobe_indices.
105	from src.core.engine import Spritesheet	Mengimpor kelas Spritesheet secara lokal.
	player.sheet =	
106	Spritesheet(SKIN_OPTIONS[wardrobe_indices[0]], 8, 8, scale=2.0)	Memuat ulang warna kulit tubuh dasar pemain.
107	outfit_path = OUTFIT_OPTIONS[wardrobe_indices[1]]	Mendapatkan path file baju terpilih.
108-113	if outfit_path: ... else: ...	Memuat baju baru jika terpilih, atau mengosongkannya jika memilih bare/tanpa baju.
114-118	hair_path = HAIR_OPTIONS[wardrobe_indices[2]] ...	Memuat gaya rambut baru atau mengeset botak gundul jika None.
119-123	hat_path = HAT_OPTIONS[wardrobe_indices[3]] ...	Memuat gaya topi baru atau melepas topi jika None.

124	update_player_wardrobe()	Memanggil fungsi kustomisasi awal untuk memuat visual default pemain saat masuk.
125	while True:	Loop Utama Game (Game Loop).
126	dt = clock.tick(60) / 16.67	Membatasi framerate ke 60 FPS dan menghitung delta waktu pendorong fisika gerakan.
127	events = pygame.event.get()	Mengambil seluruh kejadian/event antrean input di sistem.
128	gesture_thread.in_minigame = minigame_manager.in_minigame	Menyelaraskan status minigame ke thread kamera agar filter algoritma deteksi gestur tangan otomatis berubah.
129	current_g = gesture_thread.current_gesture	Membaca gestur tangan terpopuler dari kamera saat ini.
130	h_pos = gesture_thread.hand_pos	Membaca koordinat filtered ujung jari telunjuk dari thread.
131	if wardrobe_active:	BLOK KONTROL MENU WARDROBE AKTIF.
132-136	for event in events: if event.type == pygame.QUIT: ... if event.type == pygame.KEYDOWN: if event.key == pygame.K_UP: ...	Menangani penutupan program game secara bersih (menghentikan thread kamera terlebih dahulu).
137-141	elif event.key == pygame.K_LEFT: ...	Menavigasi pilihan kategori kustomisasi ke atas atau ke bawah.
142-150	elif event.key == pygame.K_RIGHT: ...	Menekan tombol arah kiri memotong indeks pilihan pakaian (mundur ke opsi sebelumnya) dan memperbarui pakaian pemain.
151-161	elif event.key == pygame.K_RETURN: ...	Menekan tombol arah kanan menambah indeks pilihan pakaian (maju ke opsi baru).
162-163	wardrobe_active = False	Menekan Enter menutup menu kustomisasi dan kembali ke Sanctuary.
164-169	game_logic.draw_ground(screen, camera) ...	Merender dunia Sanctuary di balik layar menu kustomisasi wardrobe.
170-176	overlay_w, overlay_h = 720, 440 ...	Menggambar latar belakang transparan menu kustomisasi wardrobe di tengah layar.
177-183	font_title = pygame.font.SysFont("Arial", 22, bold=True) ...	Merender judul menu kustomisasi wardrobe dan garis pembatas atas.
184-188	pygame.draw.rect(screen, (20, 30, 40, 255), (ox + 30, oy + 65, 180, 290) ...	Menggambar panel bingkai preview visual karakter sebelah kiri.
189-202	preview_timer = getattr(player, 'preview_timer', 0) + 1 ...	Menghitung frame langkah kaki karakter preview dan menggambar sprite berskala besar (160x160px) pemain yang sedang berjalan di panel preview.
203-234	categories = [...]	Merender daftar 4 kategori pakaian dengan sorotan warna cyan cerah pada opsi yang sedang aktif dipilih.
235-238	help_text = ...	Menggambar petunjuk navigasi tombol keyboard di bagian bawah panel wardrobe.
239-240	pygame.display.flip(); continue	Menyegarkan frame layar menu kustomisasi dan lewat game loop utama.
241	if upgrade_shop_active:	BLOK KONTROL MENU UPGRADE SENJATA AKTIF.

242-246	<pre> for event in events: if event.type == pygame.QUIT: ... if event.type == </pre>	Penutupan aman program.
247-251	<pre> pygame.KEYDOWN: if event.key == pygame.K_UP: ... </pre>	Menavigasi sorotan kategori upgrade senjata ke atas/bawah.
252-259	<pre> elif event.key == pygame.K_RETURN: ... </pre>	Membeli upgrade level senjata jika level saat ini < 3 dan saldo Bronze mencukupi. Saldo akan dikurangi dan memicu visual flash monitor.
260-261	<pre> elif event.key in [pygame.K_ESCAPE, pygame.K_BACKSPACE, pygame.K_q]: </pre>	Menutup menu stasiun upgrade jika menekan ESC, Backspace, atau Q.
262-267	<pre> game_logic.draw_ground(screen, ...) </pre>	Merender dunia Sanctuary di latar belakang menu upgrade.
268-274	<pre> overlay_w, overlay_h = 750, 450 ... </pre>	Menggambar panel luar transparan menu stasiun upgrade senjata berwarna ungu kemerahan.
275-285	<pre> title_surf = ... </pre>	Merender judul toko upgrade dan total saldo koin pemain di pojok kanan atas.
286-295	<pre> pygame.draw.rect(screen, (30, 20, 40) ... </pre>	Menggambar bingkai preview senjata di sebelah kiri menu.
296-305	<pre> stat_y = oy + 280 ... </pre>	Merender statistik persenan angka aktual senjata (Damage, Mag, Reload speed, Firerate) di bawah gambar preview.
306-311	<pre> categories_info = [...] </pre>	Mendefinisikan rincian data upgrade (nama, key, kalkulator perbandingan nilai, keterangan bonus upgrade).
312-328	<pre> for idx, (cat_name, cat_key, val_func, bonus_desc) in enumerate(categories_info): </pre>	Merender baris pilihan upgrade. Pilihan aktif ditandai warna merah muda cerah berpendar.
329-331	<pre> title_txt = f"{cat_name} [Lv {lvl}/3]" ... </pre>	Menampilkan nama statistik upgrade beserta level upgrade saat ini.
332-341	<pre> if lvl < 3: ... </pre>	Menampilkan biaya harga upgrade. Warna teks harga akan berwarna hijau jika saldo cukup, dan berwarna merah jika uang kurang.
342-345	<pre> else: ... </pre>	Menampilkan label hijau "SELESAI UPGRADE (MAX)" jika level statistik sudah berada di level 3.
346-351	<pre> help_text = ... ; pygame.display.flip(); continue </pre>	Menampilkan petunjuk tombol navigasi upgrade, segarkan layar monitor, dan lompat ke frame berikutnya.
352	<pre> interact_pressed = False </pre>	Mereset status pemicu interaksi Enter ke False pada loop utama.
353	<pre> if minigame_manager.in_minigame: </pre>	LOGIKA JIKA SEDANG DI DALAM MINIGAME (CITY UNDER ATTACK).
354-358	<pre> for event in events: if event.type == pygame.QUIT: ... elif event.type == </pre>	Penutupan aman program.
359-360	<pre> pygame.MOUSEMOTION: smoothed_hx, smoothed_hy = event.pos </pre>	Membaca cursor mouse fisik pemain jika bermain menggunakan mouse.

361	if minigame_manager.active_game:	Memeriksa ketersediaan objek game sebelum meneruskan event keyboard/mouse fisik ke sistem event minigame.
362	if minigame_manager.active_game is None: continue	Penanganan Crash Keluar Awal: Jika pemain menekan ESC/Q untuk keluar lebih awal, loop frame akan dilewati agar tidak terjadi crash AttributeError.
363-364	if current_g != "None": target_hx = int(h_pos[0] * WIDTH) ...	Menghitung koordinat piksel bidikan target dari rasio normalisasi MediaPipe (0.0 s.d 1.0) dikalikan lebar/tinggi layar.
365-367	smoothed_hx = smoothed_hx + (target_hx - smoothed_hx) * 0.40 ...	Smooth Lerp Filter: Meredam gerakan kursor sebesar 40% agar gerakan tangan goyah diredam menjadi halus.
368-369	fake_move = pygame.event.Event(...)	Membuat event pergerakan kursor buatan dan mengirimkannya ke minigame agar retikel kursor bidikan bergeser.
370-371	if gesture_thread.recoil_active:	Jika sinyal kejutan tembakan (recoil/sentakan telunjuk) terdeteksi aktif di thread kamera.
372	minigame_manager.active_game.handle_event(data)	Membuat event klik kiri mouse buatan untuk memicu tembakan peluru senjata di minigame.
373-374	if minigame_manager.active_game:	Memastikan objek minigame masih aktif sebelum memproses pembaruan logika waktu dan penggambaran grafis.
375-378	cam_surf = gesture_thread.frame ...	Merender layar monitor feed webcam kecil berukuran 160x120px di pojok kanan atas minigame, beserta garis tepi cyan dan label teks nama gestur terdeteksi (AIM, PISTOL, atau FIST).
379-380	pygame.display.flip(); continue	Segarkan layar game loop, lewati pemrosesan alur Sanctuary.
381-386	for event in events: if event.type == pygame.QUIT: ... if event.type ==	LOGIKA EKSPLORASI SANCTUARY: Tangani event penutupan game.
387-389	pygame.KEYDOWN: if dialogue.active: ... dialogue.next_message()	Jika dialogue box aktif, menekan tombol Enter akan melangkah ke kalimat dialog selanjutnya.
390-392	else: if event.key == pygame.K_RETURN: interact_pressed = True	Jika dialog tertutup, menekan tombol Enter menyalakan status pemicu interaksi aksi (interact_pressed = True).
393-397	elif event.key in [pygame.K_1, pygame.K_2, pygame.K_3, pygame.K_4]:	Menekan tombol angka 1-4 untuk mengganti senjata aktif eksplorasi. Mengharuskan akumulasi point_kill >= 10.000 untuk UZI, dan >= 20.000 untuk SCAR. Jika terkunci, tampilkan teks melayang peringatan berwarna merah.
398	keys = InputProxy(pygame.key.get_pressed())	Mengambil status tombol keyboard dibungkus input proxy.
399-402	if current_g == "ATAS": keys.overrides[pygame.K_UP] = True ...	Simulasi Gerak Gestur ML: Menerjemahkan gestur tangan ML (ATAS, BAWAH, KIRI, KANAN) ke tombol navigasi keyboard buatan.

403	<pre> elif current_g in ["PISTOL", "AMBIL", "ENTER"]: interact_pressed = True </pre>	Gestur "AMBIL" / "PISTOL" / "ENTER" secara otomatis memicu interaksi aksi eksplorasi.
404-406	<pre> if not dialogue.active: player.update(keys, map_size=(2560, 1440)) ... </pre>	Jika tidak sedang membaca dialog: update pergerakan pemain dan arahkan kamera gulir mengikuti pemain.
407	<pre> signal = game_logic.update(player, interact_pressed) </pre>	Memproses logika interaksi Sanctuary (api unggun, kasur medis, NPC, helipad, wardrobe, stasiun upgrade).
408-410	<pre> if signal == "START_SHOOTING": </pre>	Jika menerima sinyal pertempuran dari Kapten Jaka.
411	<pre> minigame_manager.start_minigame(ShootingRangeUltimate) </pre>	Luncurkan minigame " City Under Attack " (ShootingRangeUltimate)
412-413	<pre> elif signal == "AWARD_150_BRONZE": currency.add_bronze(150) </pre>	Block senjata yang sedang aktif dipilih saat ini. Jika menerima sinyal klaim peti suplai helipad, tambahkan 150 koin Bronze.
414-415	<pre> elif signal == "OPEN_WARDROBE": wardrobe_active = True </pre>	Buka menu pakaian jika pemain berinteraksi dengan loker.
416-417	<pre> elif signal == "OPEN_UPGRADE_SHOP": upgrade_shop_active = True game_logic.draw_ground(screen, camera) ... </pre>	Buka menu upgrade senjata jika pemain berinteraksi dengan komputer stasiun upgrade.
418-422	<pre> hud.draw(screen, player, 100) </pre>	Merender peta dasar, entitas Sanctuary (NPC, partikel, peti), sprite pemain, flash efek, dialog, dan HUD koin/nyawa di layar.
423-427	<pre> status = game_logic.get_status_text() ... </pre>	Merender panel HUD status bawah berbingkai cyan setinggi 75px, menampilkan teks status objektif, total point_kill & status kunci UZI/SCAR, serta senjata siap pakai.
428-432	<pre> cam_surf = gesture_thread.frame ... </pre>	Merender monitor kamera feed webcam kecil 160x120px beserta label nama gesturnya di pojok kanan atas layar Sanctuary.
433	<pre> pygame.display.flip() </pre>	Menyegarkan buffer grafis layar utama Pygame (<i>Double Buffering</i>).
434-436	<pre> if __name__ == "__main__": main() </pre>	Memeriksa apakah file dijalankan langsung sebagai program utama, lalu mengeksekusi fungsi main().

Hubungan Antarkomponen Utama

- **Interaksi GestureThread & main.py:** Thread gestur kamera berjalan mandiri secara asinkron. Di dalam game loop, main.py membaca gestur saat ini (current_g) untuk menggerakkan pemain (mode Sanctuary) atau membidik retikel kursor (mode Minigame).
- **Interaksi SanctuaryLogic & main.py:** main.py memanggil update status Sanctuary dan menangani perpindahan koordinat pemain atau pemberian hadiah Bronze berdasarkan sinyal feedback yang dikembalikan (OPEN_WARDROBE, AWARD_150_BRONZE, dll).
- **Interaksi MiniGameManager & main.py:** Manager memisahkan logika loop Sanctuary dengan Minigame. Ketika minigame aktif, manager langsung membajak proses rendering dan event polling untuk dialihkan ke ShootingRangeUltimate. Setelah waktu habis, manager keluar kembali ke loop Sanctuary dan memberikan Bronze hasil eliminasi zombie.

Penjelasan Kode: sanctuary_logic.py

Dokumen ini berisi penjelasan detail baris demi baris untuk file /home/abuyyy/PemogramanLanjut/src/core/sanctuary_logic.py. Penjelasan ini ditulis dalam bahasa Indonesia untuk membantu persiapan demo program.

Deskripsi & Tujuan File

File sanctuary_logic.py mengatur seluruh perilaku simulasi lingkungan, partikel visual, dialog cerita, dan interaksi pemain di area pangkalan aman ("The Sanctuary"): 1. **Partikel Lingkungan (SanctuaryParticle)**: Percikan api unggun, pendaran hijau stasiun medis, cipratan air kolam, dan debu pendaratan kotak suplai. 2. **Karakter Non-Pemain (NPC)**: Penatua Aris, Kapten Jaka, dan Warga Sipil yang ditempatkan secara teratur di koordinat peta. 3. **Mekanika Interaksi Lingkungan**: - Efek hangat/terbakar api unggun. - Pemulihan otomatis di area kasur medis. - Hambatan air dalam di kolam. - Penggunaan Loker Pakaian (Wardrobe) untuk modifikasi model sprite. - Penggunaan Stasiun Peningkatan (Upgrade Shop) untuk memperkuat statistik senjata. - Panggilan Kotak Suplai Udara (Supply Drop) di helipad dengan parasut udara dinamis dan hadiah koin. 4. **Logika Quest Pemicu Pertempuran**: Pemain harus berbicara dengan Penatua Aris sebelum dapat meyakinkan Kapten Jaka untuk mengaktifkan alarm pertahanan kota, memindahkan posisi ke menara, dan memicu peluncuran minigame.

Penjelasan Baris Demi Baris

Berikut adalah tabel analisis baris demi baris dari kode sumber sanctuary_logic.py:

Baris	Kode Sumber	Penjelasan
1	import pygame	Mengimpor pustaka Pygame untuk menangani grafik, bentuk geometris, font, dan input game.
2	import math	Mengimpor modul matematika untuk menghitung jarak Euclidean menggunakan hypot dan kalkulasi trigonometri.
3	import random	Mengimpor modul random untuk menghasilkan posisi partikel acak dan memilih dialog acak.
4	class SanctuaryParticle:	Deklarasi kelas SanctuaryParticle untuk membuat visual efek partikel kecil.
5	def __init__(self, x, y, color, vx=None, vy=None, size=None):	Konstruktur partikel menerima koordinat awal, warna, kecepatan horizontal/vertikal opsional, dan ukuran opsional.
6	self.pos = [x, y]	Menyimpan posisi partikel dalam list [x, y].
7	self.vel = [vx if vx is not None else random.uniform(-1, 1), vy if vy is not None else random.uniform(-1.5, -0.5)]	Menetapkan kecepatan gerak partikel (default horizontal acak -1 s.d 1, vertikal ke atas -1.5 s.d -0.5).
8	self.color = color	Menyimpan warna partikel (RGB).
9	self.size = size if size is not None else random.randint(3, 6)	Mengatur ukuran partikel awal (default acak 3 s.d 6 piksel).
10	self.life = 1.0	Menetapkan sisa masa hidup partikel awal sebesar 100% (1.0).
11	self.decay = random.uniform(0.02, 0.04)	Menetapkan tingkat pengurangan masa hidup per frame secara acak antara 0.02 s.d 0.04.
12	def update(self):	Metode untuk memperbarui posisi dan masa hidup partikel di setiap frame.

13	<code>self.pos[0] += self.vel[0]</code>	Menggeser posisi X partikel berdasarkan kecepatan horizontalnya.
14	<code>self.pos[1] += self.vel[1]</code>	Menggeser posisi Y partikel berdasarkan kecepatan vertikalnya.
15	<code>self.life -= self.decay</code>	Mengurangi sisa masa hidup partikel.
16	<code>return self.life > 0</code>	Mengembalikan nilai True jika partikel masih hidup (<code>life > 0</code>), atau False jika sudah mati.
17	<code>def draw(self, screen, camera):</code>	Menggambar lingkaran partikel ke layar.
18	<code>draw_pos = camera.apply(self.pos)</code>	Menghitung koordinat layar partikel disesuaikan offset kamera scroll.
19	<code>pygame.draw.circle(screen, self.color, (int(draw_pos[0]), int(draw_pos[1])), int(self.size * self.life))</code>	Merender lingkaran dengan ukuran yang mengecil seiring waktu (<code>size * life</code>).
20	<code>PROLOGUE_M3 = [</code>	Mendefinisikan list dialog prolog pembuka game.
21	<code>"Cahaya matahari menyilaukan mataku...",</code>	Kalimat prolog baris 1.
22	<code>"Setelah berhari-hari di dalam kegelapan Metro, akhirnya aku keluar.",</code>	Kalimat prolog baris 2.
23	<code>"Tunggu... suara apa itu?",</code>	Kalimat prolog baris 3.
24	<code>"Suara tawa? Suara orang berbicara?",</code>	Kalimat prolog baris 4.
25	<code>"Selamat datang di 'The Sanctuary'. Benteng terakhir kemanusiaan."</code>	Kalimat prolog baris 5.
26	<code>]</code>	Menutup list dialog prolog.
27	<code>NPC_DIALOGUES = {</code>	Kamus dialog santai untuk NPC berdasarkan peran mereka.
28	<code>"Guard": [</code>	Daftar dialog NPC Penjaga.
29	<code>"Jaga langkahmu, kawan. Kami tidak ingin ada masalah di sini.",</code>	Opsi dialog penjaga 1.
30	<code>"Senang melihat wajah baru yang bukan zombie."</code>	Opsi dialog penjaga 2.
31	<code>],</code>	Penutup dialog penjaga.
32	<code>"Citizen": [</code>	Daftar dialog NPC Warga Sipil.
33	<code>"Katanya ada kota lain di Utara, tapi siapa yang tahu?",</code>	Opsi warga 1.
34	<code>"Anak-anak akhirnya bisa bermain tanpa rasa takut... setidaknya untuk sekarang."</code>	Opsi warga 2.
35	<code>],</code>	Penutup dialog warga.
36	<code>"Elder": [</code>	Daftar dialog NPC Tetua.
37	<code>"Selamat datang, pengembara. Kami sudah mendengar kabarmu lewat radio.",</code>	Opsi tetua 1.
38	<code>"Dunia luar sudah hancur, tapi di sini... kita mencoba membangun kembali."</code>	Opsi tetua 2.
39	<code>]</code>	Penutup dialog tetua.
40	<code>}</code>	Penutup kamus NPC_DIALOGUES.

41	class SanctuaryLogic:	Deklarasi kelas utama logika lingkungan Sanctuary.
42	def __init__(self, dialogue):	Inisialisasi logika Sanctuary dengan menyertakan objek dialog box.
43	self.dialogue = dialogue	Menyimpan referensi objek dialog box.
44	self.alarm_active = False	Status keaktifan alarm pertempuran (awal False).
45	self.elder_met = False	Status apakah pemain sudah menemui Penatua Aris (awal False).
46	self.npcs = []	List kosong untuk menampung instansiasi NPC.
47	self._setup_npcs()	Memanggil metode internal untuk membuat NPC di peta.
48	self.font = pygame.font.SysFont("monospace", 14, bold=True)	Inisialisasi font petunjuk interaksi monospace 14px tebal.
49	try:	Blok try-except untuk memuat aset gambar peta Sanctuary.
50	self.map_img = pygame.image.load("assets/images/sanctuary_map.png").convert()	Memuat gambar map latar belakang Sanctuary.
51	self.map_img = pygame.transform.scale(self.map_img, (2560, 1440))	Menskalakan ukuran gambar peta menjadi 2560x1440 piksel.
52	except Exception as e:	Menangkap kegagalan memuat gambar.
53	print(f"Error loading sanctuary map image: {e}")	Mencetak pesan error ke konsol terminal.
54	self.map_img = None	Menetapkan map_img ke None (mengaktifkan mode fallback warna rumput).
55	self.particles = []	Inisialisasi list penampung partikel aktif di area Sanctuary.
56	self.floating_texts = []	Inisialisasi list penampung teks melayang.
57	self.crate = None	Status peti suplai helipad (awal None).
58	self.crate_cooldown = 0	Waktu tunggu pemanggilan peti suplai berikutnya.
59	def _setup_npcs(self):	Metode internal membuat dan memposisikan NPC di peta.
60	from src.entities.entities import NPC	Mengimpor kelas NPC dari modul entitas terpadu.
61	self.npcs = [62 NPC(63 name="Penatua Aris", 64 x=1280, y=600, 65 role="Elder", base_path="FREE Mana Seed 66 Character Base Demo	File sprite tubuh dasar. 2.0/char_a_p1/char_a_p1_0bas_humn_v09.png", outfit_path="FREE Mana 67 Seed Character Base Demo
	2.0/char_a_p1/1out/char_a_p1_1out_fstr_v05.png", hair_path="FREE Mana Seed 68 Character Base Demo	File sprite baju. File sprite rambut. 2.0/char_a_p1/4har/char_a_p1_4har_bob1_v02.png"
69	),	Penutup NPC pertama.
70	NPC(71 name="Kapten Jaka",	Instansiasi NPC Kapten Jaka. Nama NPC.
72	x=1800, y=650,	Posisi di jalan gerbang timur.
73	role="Guard",	Peran NPC.

74	base_path="FREE Mana Seed Character Base Demo 2.0/char_a_p1/char_a_p1_0bas_humn_v00.png", outfit_path="FREE Mana	File sprite tubuh dasar.
75	Seed Character Base Demo 2.0/char_a_p1/1out/char_a_p1_1out_fstr_v01.png", hair_path="FREE Mana Seed	File sprite baju penjaga.
76	Character Base Demo 2.0/char_a_p1/4har/char_a_p1_4har_dap1_v08.png"	File sprite rambut.
77	),	Penutup NPC kedua.
78	NPC(	Instansiasi NPC Warga Sipil.
79	name="Warga Sipil",	Nama NPC.
80	x=400, y=1100,	Posisi di area medis sebelah barat.
81	role="Citizen",	Peran NPC.
82	base_path="FREE Mana Seed Character Base Demo 2.0/char_a_p1/char_a_p1_0bas_humn_v02.png", outfit_path="FREE Mana	File sprite tubuh dasar.
83	Seed Character Base Demo 2.0/char_a_p1/1out/char_a_p1_1out_pfpn_v02.png", hair_path="FREE Mana Seed	File sprite baju warga.
84	Character Base Demo 2.0/char_a_p1/4har/char_a_p1_4har_bob1_v10.png"	File sprite rambut.
85	)	Penutup NPC ketiga.
86	]	Penutup list self.npcs.
87	def update(self, player, interact_pressed):	Metode update logika lingkungan per frame di area Sanctuary.
88	for npc in self.npcs:	Melakukan perulangan untuk memperbarui AI pergerakan acak NPC.
89	npc.update(map_size=(2560, 1440))	Menggerakkan NPC dalam batas peta Sanctuary.
90	self.particles = [p for p in self.particles if p.update()]	Memperbarui partikel aktif dan menghapus partikel yang masa hidupnya habis.
91	for ft in self.floating_texts:	Melakukan perulangan untuk memperbarui posisi teks melayang.
92	ft['pos'][1] -= 0.6	Menggeser teks melayang ke atas (Y berkurang 0.6px per frame).
93	ft['timer'] -= 1	Mengurangi sisa durasi tampil teks melayang.
94	self.floating_texts = [ft for ft in self.floating_texts if ft['timer'] > 0]	Menyaring teks melayang yang sisa durasinya masih di atas 0.
95	if self.crate_cooldown > 0:	Jika cooldown peti suplai aktif.
96	self.crate_cooldown -= 1	Kurangi cooldown peti suplai sebanyak 1 per frame.
97	px, py = player.pos[0], player.pos[1]	Menyalin posisi koordinat pemain saat ini ke variabel lokal.
98	player.speed_multiplier = 0.40	Membatasi kecepatan berjalan pemain di Sanctuary secara default ke 40% agar santai.
99	campfire_dist = math.hypot(px - 1220, py - 560)	Menghitung jarak pemain ke pusat api unggun di koordinat (1220, 560).
100	if campfire_dist < 120:	Jika pemain berada di sekitar jangkauan api unggun (< 120px).
101	if campfire_dist < 30:	Jika pemain terlalu dekat/menginjak api unggun (< 30px).
102	player.health = max(0.0, player.health - 0.7)	Mengurangi darah pemain sebesar 0.7 per frame (terbakar).
103	player.injured = True	Mengeset status cedera terluka menjadi True.

104	<code>if random.random() < 0.15:</code>	Peluang 15% memunculkan indikator teks melayang terbakar.
105	<code>self.floating_texts.append({'text': "BURNING! -HP", 'pos': [px + random.randint(-15, 15), py - 20], 'timer': 20, 'color': (255, 50, 50)})</code>	Menambahkan teks melayang merah "BURNING! -HP".
106	<code>if random.random() < 0.4:</code>	Peluang 40% mengeluarkan partikel api merah oranye di kaki pemain.
107	<code>self.particles.append(SanctuaryParticle(px + random.uniform(-10, 10), py + 10, (255, random.randint(50, 150), 0)))</code>	Menambahkan partikel api menyala.
108	<code>else:</code>	Jika berada di zona hangat aman (jarak 30px s.d 120px).
109	<code>if player.adrenaline_timer <= 0:</code>	Jika efek adrenalin belum aktif.
110	<code>self.floating_texts.append({'text': "WARMED BY CAMPFIRE (Speed Boost!)", 'pos': [px, py - 30], 'timer': 45, 'color': (255, 150, 0)})</code>	Menampilkan teks melayang oranye penanda status peningkatan kecepatan.
111	<code>player.adrenaline_timer = 120</code>	Memberikan efek status adrenalin selama 120 frame (~2 detik).
112	<code>if random.random() < 0.1:</code>	Peluang 10% memunculkan pendaran partikel percikan api kecil.
113	<code>self.particles.append(SanctuaryParticle(px + random.uniform(-10, 10), py + 10, (255, 180, 50), vy=random.uniform(-1.0, -0.3)))</code>	Menambahkan partikel percikan api ringan.
114	<code>if 100 < px < 550 and 900 < py < 1350:</code>	Memeriksa apakah pemain berada di zona kasur medis sebelah barat daya.
115	<code>if player.health < 100:</code>	Jika darah pemain kurang dari 100%.
116	<code>player.health = min(100.0, player.health + 0.3)</code>	Memulihkan darah pemain sebesar 0.3 per frame.
117	<code>player.injured = False</code>	Menghapus status cedera terluka pemain.
118	<code>if random.random() < 0.08:</code>	Peluang 8% menampilkan indikator teks melayang pemulihan.
119	<code>self.floating_texts.append({'text': "+HEAL", 'pos': [px + random.randint(-15, 15), py - 20], 'timer': 25, 'color': (100, 255, 100)})</code>	Menambahkan teks melayang hijau "+HEAL".
120	<code>if random.random() < 0.15:</code>	Peluang 15% memunculkan partikel pendaran hijau stasiun medis.
121	<code>self.particles.append(SanctuaryParticle(px + random.uniform(-15, 15), py + 15, (50, 255, 50), size=random.randint(3, 5)))</code>	Menambahkan partikel hijau pemulihan.
122	<code>if 1950 < px < 2400 and 900 < py < 1300:</code>	Memeriksa apakah pemain berjalan masuk ke kolam air sebelah tenggara.
123	<code>player.speed_multiplier = 0.20</code>	Memotong kecepatan gerak pemain menjadi hanya 20% karena hambatan air.

124	is_moving = player.state in ['walk', 'run']	Cek apakah pemain sedang dalam keadaan bergerak.
125	if is_moving and random.random() < 0.35:	Jika bergerak di air, peluang 35% memicu cipratan air.
126	self.particles.append(SanctuaryParticle(px + random.uniform(-15, 15), py + 15, (100, 180, 255), vx=random.uniform(-1.2, 1.2), vy=random.uniform(-0.4, 0.4)))	Menambahkan partikel percikan air berwarna biru muda.
127	helipad_dist = math.hypot(px - 430, py - 410)	Menghitung jarak pemain ke pusat landasan helipad di koordinat (430, 410).
128	signal_to_return = None	Inisialisasi variabel kembalian sinyal dengan None.
129	if helipad_dist < 60:	Jika pemain berdiri di dekat helipad (< 60px).
130	if interact_pressed:	Jika tombol interaksi ditekan.
131	if not self.crate and self.crate_cooldown <= 0:	Jika tidak ada peti suplai aktif dan cooldown sudah selesai.
132	self.crate = {'pos': [430, 410], 'z_height': 500, 'landed': False, 'opened': False}	Menjatuhkan peti suplai baru di helipad pada ketinggian awal 500px di udara.
133	self.floating_texts.append({'text': "SUPPLY DROP CALLED!", 'pos': [px, py - 30], 'timer': 60, 'color': (0, 255, 255)})	Menampilkan teks melayang cyan pemicu peti suplai.
134	elif self.crate and self.crate['landed'] and not self.crate['opened']:	Jika peti sudah mendarat di tanah dan belum dibuka pemain.
135	self.crate['opened'] = True	Mengubah status peti menjadi telah dibuka.
136	self.floating_texts.append({'text': "+\$150 BRONZE CURRENCY", 'pos': [px, py - 30], 'timer': 60, 'color': (255, 215, 0)})	Menampilkan hadiah uang di layar.
137	self.crate_cooldown = 1800	Memasang cooldown pemanggilan peti selama 1800 frame (~30 detik).
138	signal_to_return = "AWARD_150_BRONZE"	Mengembalikan sinyal pemberian hadiah koin perunggu ke engine utama.
139	if self.crate:	Jika objek peti suplai ada.
140	if not self.crate['landed']:	Jika peti masih melayang di udara (belum mendarat).
141	self.crate['z_height'] -= 10	Mengurangi ketinggian peti sebesar 10px per frame (turun vertikal).
142	if self.crate['z_height'] <= 0:	Jika menyentuh tanah (ketinggian <= 0).
143	self.crate['z_height'] = 0	Mengunci ketinggian di 0.
144	self.crate['landed'] = True	Mengeset status mendarat menjadi True.
145	for _ in range(20):	Melakukan looping 20 kali untuk memicu efek pendaratan.
146	self.particles.append(SanctuaryParticle(430, 410, (160, 160, 160), vx=random.uniform(-2.5, 2.5), vy=random.uniform(-1.5, 1.5)))	Mengeluarkan partikel abu-abu debu tanah mengepul di sekitar peti.
147	if interact_pressed:	Jika pemain menekan tombol interaksi di dekat NPC.

148	for npc in self.npcs:	Melakukan iterasi di daftar NPC.
	dist =	
149	math.hypot(player.pos[0] - npc.pos[0], player.pos[1] - npc.pos[1])	Menghitung jarak pemain ke NPC.
150	if dist < 100:	Jika pemain berada di dalam radius interaksi dekat NPC (< 100px).
151	if npc.name == "Penatua Aris":	Jika berbicara dengan Penatua Aris.
152	self.elder_met = True	Menyetel status pertemuan Penatua menjadi True.
	self.dialogue.show(["PENATUA ARIS: Selamat datang. Kota ini dalam bahaya.", "Bicara pada Kapten Jaka di gerbang jalan itu!"])	Membuka kotak dialog cerita dan memberi instruksi menemui Kapten Jaka.
153	elif npc.name == "Kapten Jaka":	Jika berbicara dengan Kapten Jaka.
154	if self.elder_met:	Jika pemain sudah bertemu dengan Penatua Aris sebelumnya.
155	self.alarm_active = True	Aktifkan alarm kota pertahanan.
156	player.pos = [2500, 300]	Teleportasi posisi pemain ke dekat pagar pertahanan (2500, 300).
157	self.dialogue.show(["KAPTEN JAKA: Zombie datang! Cepat ke menara!", "KITA HARUS BERTAHAN!"])	Membuka dialog darurat ancaman zombie.
158		
159	return "START_SHOOTING"	Mengembalikan sinyal "START_SHOOTING" ke loop utama untuk memulai minigame.
160	else:	Jika belum berbicara dengan Penatua Aris.
	self.dialogue.show(["KAPTEN JAKA: Cari Penatua Aris dulu! Dia di sebelah barat."])	Memberi arahan untuk menemui Penatua Aris terlebih dahulu.
161		
162	else:	Jika berbicara dengan NPC acak lainnya (Warga Sipil).
163	role = npc.role	Mengambil peran NPC.
	msg =	
164	random.choice(NPC_DIALOGUES.get(role, ["Halo!"])))	Mengambil dialog santai acak dari database dialog NPC.
165	self.dialogue.show([f"{npc.name}": {msg}"])	Membuka dialog box menampilkan teks ucapan warga.
166	return None	Membatalkan interaksi ganda pada frame yang sama.
167	w_dist = math.hypot(px - 145, py - 875)	Menghitung jarak pemain ke loker pakaian (Wardrobe) di koordinat (145, 875).
168	if w_dist < 60:	Jika berada dekat loker wardrobe (< 60px).
169	if interact_pressed:	Jika menekan tombol interaksi.
	signal_to_return =	
170	"OPEN_WARDROBE"	Mengeset sinyal kembalian ke "OPEN_WARDROBE" untuk membuka menu kostum.
171	upg_dist = math.hypot(px - 1500, py - 850)	Menghitung jarak pemain ke komputer stasiun upgrade di koordinat (1500, 850).
172	if upg_dist < 60:	Jika berada dekat komputer stasiun upgrade (< 60px).
173	if interact_pressed:	Jika menekan tombol interaksi.
	signal_to_return =	
174	"OPEN_UPGRADE_SHOP"	Mengeset sinyal kembalian ke "OPEN_UPGRADE_SHOP" untuk membuka menu upgrade senjata.

175	<code>return signal_to_return</code>	Mengembalikan sinyal interaksi aktif yang terdeteksi ke loop game utama.
176	<code>def draw_ground(self, screen, camera):</code>	Menggambar grafis tanah/peta dasar Sanctuary.
177	<code>if self.map_img:</code>	Jika gambar peta Sanctuary berhasil dimuat.
178	<code>screen.blit(self.map_img, (camera.camera.x, camera.camera.y))</code>	Menggambar background peta dengan penyesuaian posisi kamera gulir.
179	<code>else:</code>	Jika gambar peta gagal dimuat.
180	<code>screen.fill((30, 100, 30))</code>	Fallback: Warnai latar belakang hijau rumput pekat.
181	<code>pygame.draw.rect(screen, (100, 80, 50), camera.apply(pygame.Rect(950, 0, 300, 4000)))</code>	Fallback: Menggambar jalan tanah cokelat vertikal di peta.
182	<code>def draw_entities(self, screen, camera, player=None):</code>	Menggambar objek/entitas berlapis di atas tanah Sanctuary.
183	<code>for p in self.particles:</code>	Melakukan iterasi di list partikel aktif.
184	<code>p.draw(screen, camera)</code>	Merender gambar lingkaran partikel ke layar.
185	<code>if self.crate and not self.crate['opened']:</code>	Jika peti suplai aktif di peta dan belum dibuka.
186	<code>cx, cy = self.crate['pos']</code>	Mengambil koordinat X dan Y peti suplai di peta.
187	<code>screen_pos = camera.apply((cx, cy))</code>	Mengonversi koordinat peti di peta menjadi koordinat layar.
188	<code>if not self.crate['landed']:</code>	Jika peti sedang melayang jatuh.
189	<code>shadow_r = int(15 * (1.0 - self.crate['z_height'] / 500.0))</code>	Menghitung radius bayangan elips di tanah (membesar seiring turunnya peti).
190	<code>shadow_surf = pygame.Surface((shadow_r * 2, shadow_r), pygame.SRCALPHA)</code>	Membuat permukaan transparan untuk bayangan.
191	<code>pygame.draw.ellipse(shadow_surf, (0, 0, 0, 100), (0, 0, shadow_r * 2, shadow_r))</code>	Menggambar bayangan elips hitam transparan di tanah.
192	<code>screen.blit(shadow_surf, (screen_pos[0] - shadow_r, screen_pos[1] - shadow_r // 2))</code>	Menempelkan bayangan tepat di bawah peti.
193	<code>crate_y = screen_pos[1] - self.crate['z_height']</code>	Menghitung posisi Y rendering peti di layar (dikurangi tinggi jatuh).
194	<code>pygame.draw.rect(screen, (120, 60, 10), (screen_pos[0] - 16, crate_y - 16, 32, 32), border_radius=4)</code>	Menggambar tepi luar peti cokelat tua.
195	<code>pygame.draw.rect(screen, (180, 130, 40), (screen_pos[0] - 12, crate_y - 12, 24, 24), border_radius=2)</code>	Menggambar isi dalam peti cokelat muda.
196	<code>pygame.draw.line(screen, (220, 220, 220), (screen_pos[0] - 16, crate_y - 16), (screen_pos[0] - 25, crate_y - 45), 2)</code>	Menggambar tali parasut sebelah kiri.

197	<pre> pygame.draw.line(screen, (220, 220, 220), (screen_pos[0] + 16, crate_y - 16), (screen_pos[0] + 25, crate_y - 45), 2) </pre>	Menggambar tali parasut sebelah kanan.
198	<pre> pygame.draw.arc(screen, (240, 240, 240), (screen_pos[0] - 30, crate_y - 65, 60, 40), 0, math.pi, 3) </pre>	Menggambar kubah parasut putih di atas peti.
199	<pre> else: pygame.draw.rect(screen, (120, 60, 10), </pre>	Jika peti sudah mendarat di tanah.
200	<pre> (screen_pos[0] - 16, screen_pos[1] - 16, 32, 32), border_radius=4) pygame.draw.rect(screen, (180, 130, 40), </pre>	Menggambar tepi luar peti di atas tanah.
201	<pre> (screen_pos[0] - 12, screen_pos[1] - 12, 24, 24), border_radius=2) </pre>	Menggambar isi dalam peti di tanah.
202	<pre> if player: </pre>	Memeriksa objek pemain.
203	<pre> p_dist = math.hypot(player.pos[0] - cx, player.pos[1] - cy) </pre>	Menghitung jarak pemain ke peti di tanah.
204	<pre> if p_dist < 60: </pre>	Jika pemain berada sangat dekat dengan peti (< 60px).
205	<pre> hint = self.font.render("PRESS ENTER TO CLAIM SUPPLY", True, (0, 255, 255)) screen.blit(hint, </pre>	Membuat permukaan teks petunjuk cyan.
206	<pre> (screen_pos[0] - hint.get_width() // 2, screen_pos[1] - 35)) if player and not </pre>	Menempelkan petunjuk tepat di atas peti suplai.
207	<pre> self.crate and self.crate_cooldown <= 0: h_dist = </pre>	Jika helipad kosong dan siap menerima panggilan drop.
208	<pre> math.hypot(player.pos[0] - 430, player.pos[1] - 410) </pre>	Menghitung jarak pemain ke landasan helipad.
209	<pre> if h_dist < 60: </pre>	Jika berada sangat dekat helipad (< 60px).
210	<pre> h_pos = camera.apply((430, 410)) hint = </pre>	Menghitung posisi koordinat layar helipad.
211	<pre> self.font.render("PRESS ENTER TO CALL SUPPLY DROP", True, (0, 255, 255)) screen.blit(hint, </pre>	Membuat teks petunjuk cyan.
212	<pre> (h_pos[0] - hint.get_width() // 2, h_pos[1] - 25)) if player: </pre>	Menempelkan petunjuk di atas helipad.
213	<pre> w_dist = </pre>	Memeriksa objek pemain.
214	<pre> math.hypot(player.pos[0] - 145, player.pos[1] - 875) </pre>	Menghitung jarak pemain ke loker wardrobe.
215	<pre> if w_dist < 60: </pre>	Jika berada dekat loker wardrobe (< 60px).
216	<pre> w_pos = camera.apply((145, 875)) </pre>	Menghitung posisi koordinat layar loker pakaian.

217	<pre> hint = self.font.render("PRESS ENTER TO USE WARDROBE", True, (0, 255, 255)) screen.blit(hint, (w_pos[0] - </pre>	Membuat teks petunjuk pemakaian loker pakaian.
218	<pre> hint.get_width() // 2, w_pos[1] - 25)) </pre>	Menempelkan petunjuk di atas loker pakaian.
219	<pre> upg_pos = camera.apply((1500, 850)) </pre>	Menghitung posisi koordinat layar komputer stasiun upgrade (1500, 850).
220	<pre> pygame.draw.rect(screen, (50, 60, 70), (upg_pos[0] - 20, upg_pos[1] - 30, 40, 60), border_radius=5) pygame.draw.rect(screen, (0, 255, 255), (upg_pos[0] - 20, upg_pos[1] - 30, 40, 60), 2, border_radius=5) </pre>	Menggambar badan luar konsol stasiun berwarna abu-abu biru dengan sudut membulat.
221	<pre> pygame.draw.rect(screen, (10, 20, 30), (upg_pos[0] - 14, upg_pos[1] - 22, 28, 20)) </pre>	Menggambar garis tepi konsol berwarna cyan neon terang.
222	<pre> pygame.draw.rect(screen, (0, 200, 200), (upg_pos[0] - 14, upg_pos[1] - 22, 28, 20), 1) </pre>	Menggambar layar gelap monitor komputer konsol.
223	<pre> pulse = (pygame.time.get_ticks() // 400) % 2 screen_color = (0, 255, 120) if pulse else (0, 150, 80) </pre>	Menggambar garis tepi layar monitor komputer berwarna cyan redup.
224	<pre> pygame.draw.circle(screen, screen_color, (upg_pos[0] - 5, upg_pos[1] - 12), 2) </pre>	Membuat nilai modulo 2 berdasarkan waktu sistem untuk efek denyut lampu indikator.
225	<pre> pygame.draw.circle(screen, (255, 50, 50), (upg_pos[0] + 5, upg_pos[1] - 12), 2) </pre>	Memilih warna hijau berkedip (hijau terang jika pulse=1, hijau gelap jika pulse=0).
226	<pre> pygame.draw.rect(screen, (30, 35, 40), (upg_pos[0] - 22, upg_pos[1] - 2, 44, 8), border_radius=2) </pre>	Menggambar lampu indikator kiri yang berkedip hijau di layar monitor.
227	<pre> if player: upg_dist = math.hypot(player.pos[0] - 1500, player.pos[1] - 850) </pre>	Menggambar lampu indikator kanan berwarna merah statis.
228	<pre> if upg_dist < 60: </pre>	Menggambar keyboard fisik konsol di bagian bawah layar stasiun upgrade.
229	<pre> hint = self.font.render("PRESS ENTER TO UPGRADE WEAPONS", True, (0, 255, 255)) screen.blit(hint, (upg_pos[0] - </pre>	Memeriksa objek pemain.
230	<pre> hint.get_width() // 2, upg_pos[1] - 55)) </pre>	Menghitung jarak pemain ke komputer stasiun upgrade.
231	<pre> for npc in self.npcs: </pre>	Jika pemain berdiri dekat komputer stasiun upgrade (< 60px).
232	<pre> npc.draw(screen, camera) </pre>	Membuat teks petunjuk pemakaian toko peningkatan senjata.
233	<pre> d_pos = camera.apply(npc.pos) </pre>	Menempelkan petunjuk di atas stasiun upgrade.
234		Melakukan iterasi untuk menggambar setiap NPC di Sanctuary.
235		Menggambar sprite NPC disesuaikan posisi kamera.
236		Menghitung koordinat kepala NPC di layar.

237	name_tag = self.font.render(npc.name, True, (255, 255, 255)) screen.blit(name_tag, (d_pos[0] + 16 - name_tag.get_width() // 2, d_pos[1] - 15))	Merender nama NPC berwarna putih.
238		Menampilkan label nama di atas kepala NPC dengan penyeimbang sumbu X.
239	for ft in self.floating_texts:	Melakukan iterasi untuk menggambar setiap teks melayang aktif.
240	shadow_surf = self.font.render(ft['text'], True, (0, 0, 0)) text_surf = self.font.render(ft['text'], True, ft['color'])	Merender teks bayangan berwarna hitam (RGB 0,0,0) sebagai outline.
241	sp = camera.apply(ft['pos'])	Merender teks utama berwarna sesuai kriteria partikel.
242		Menghitung koordinat layar teks melayang.
243	screen.blit(shadow_surf, (sp[0] + 1, sp[1] + 1))	Menggambar teks bayangan hitam dengan pergeseran 1px (Y+1, X+1).
244	screen.blit(text_surf, sp)	Menggambar teks utama di atas bayangannya pada posisi koordinat aslinya.
245	def get_status_text(self):	Metode mengambil status objektif cerita Sanctuary aktif saat ini.
246	if self.alarm_active: return "STATUS: PERTAHANAN AKTIF!" if not self.elder_met:	Mengembalikan status alarm pertahanan kota aktif jika terpicu.
247	return "OBJEKTIF: Cari dan bicara ke Penatua Aris (Warna Oranye)" return "OBJEKTIF: Bicara ke Kapten Jaka di jalan (Warna Biru)" (Baris Kosong)	Mengembalikan petunjuk mencari Penatua jika belum pernah bertemu.
248		Mengembalikan petunjuk bertemu Kapten Jaka jika prasyarat terpenuhi.
249		Akhir dari berkas kode sumber.
# Penjelasan Kode: shooting_range.py		

Dokumen ini berisi penjelasan detail baris demi baris untuk file /home/abuyyy/PemogramanLanjut/src/core/minigames/shooting_range.py. Penjelasan ini ditulis dalam bahasa Indonesia untuk membantu persiapan demo program.

Deskripsi & Tujuan File

File shooting_range.py mendefinisikan seluruh logika dari minigame "**City Under Attack**" (latihan menembak untuk mempertahankan kota): 1. **Konfigurasi Senjata (WeaponType & TacticalWeapon)**: Mengatur statistik dasar 4 senjata (Micro Uzi, SCAR-H, Spas-12 Shotgun, AWM Sniper) beserta sistem amunisi, penanganan delay tembakan, dan durasi isi ulang (*reload*). 2. **Sistem Deteksi Tembakan & Recoil**: Mengurangi amunisi, memutar SFX tembakan (suarapistol.mp3), memicu getaran layar (*screen shake*), dan mendeteksi tabrakan koordinat kursor (*crosshair*) dengan zombie (termasuk deteksi *headshot* / tembakan di kepala untuk melipatgandakan damage). 3. **Mekanisme Target Zombie**: Zombie akan spawn secara berkala dengan arah dan baris animasi yang acak, bergerak mendekati layar bawah, dan memiliki bar nyawa di atas kepala. Setiap kelipatan 15 eliminasi, Boss Zombie ("Crimson Abomination") raksasa akan lahir dengan kapasitas nyawa tebal dan memberikan hadiah bonus melimpah. 4. **Bantuan Bidik Otomatis (Aim Assist) & Bidikan Kunci**: Memeriksa target terdekat dengan retikel kursor, lalu mendorong kursor secara magnetis mendekat sebesar 55% ke arah kepala zombie jika berada di dalam radius sensitivitas (90px) untuk mempermudah bidikan pemain webcam.

Penjelasan Baris Demi Baris

Berikut adalah tabel penjelasan baris demi baris dari kode sumber shooting_range.py:

Baris	Kode Sumber	Penjelasan
1	<code>import pygame, math, random, time</code>	Mengimpor Pygame untuk grafik/audio, math untuk perhitungan jarak, random untuk acak, dan time untuk jeda waktu.
2	<code>from typing import List, Dict, Tuple, Optional from src.core.engine</code>	Mengimpor helper tipe data untuk petunjuk tipe statis (typing).
3	<code>import MiniGame, Spritesheet</code>	Mengimpor kelas induk MiniGame dan pembaca visual Spritesheet dari engine utama.
4	<i>(Baris Kosong)</i>	Pemisah visual antarkelas.
5	<code>class PixelPalette:</code>	Deklarasi kelas PixelPalette untuk menyimpan warna-warna tema UI game.
6	<code>GOLD, CYAN_GLOW, FIRE_GLOW, BLOOD = (255, 215, 0), (0, 255, 255), (255, 50, 50), (180, 0, 0)</code>	Konstanta warna koin emas, cyan berpendar, merah api, dan merah darah zombie.
7	<i>(Baris Kosong)</i>	Pemisah visual antarkelas.
8	<code>class Particle:</code>	Deklarasi kelas Particle untuk partikel cipratan darah ketika zombie tertembak.
9	<code>def __init__(self, x, y, color, size_range=(2, 8)):</code>	Konstruktur menerima koordinat awal partikel, warna, dan rentang ukuran opsional.
10	<code>self.pos, self.shadow = [x, y], [random.uniform(-3, 3), random.uniform(-3, 3)]</code>	Menyimpan posisi dan kecepatan sebaran horizontal/vertikal acak (antara -3 s.d 3).
11	<code>self.color, self.size, self.life = color, random.randint(*size_range), 1.0</code>	Menyimpan warna, memilih ukuran acak, dan menetapkan masa hidup awal partikel ke 100% (1.0).
12	<code>def update(self):</code>	Metode memperbarui pergerakan dan sisa masa hidup partikel di setiap frame.
13	<code>self.pos[0] += self.vel[0]; self.pos[1] += self.vel[1]; self.life -= 0.02</code>	Menggeser koordinat X/Y partikel sesuai kecepatannya, lalu mengurangi masa hidup sebesar 0.02 per frame.
14	<code>return self.life > 0</code>	Mengembalikan status True jika partikel masih hidup (life > 0), atau False jika sudah mati.
15	<code>def draw(self, screen, offset=(0,0)): pygame.draw.circle(screen, self.color, (int(self.pos[0] + offset[0]), int(self.pos[1] + offset[1])), int(self.size * self.life))</code>	Menggambarkan lingkaran partikel ke layar.
16		Menggambarkan partikel bulat disesuaikan dengan goyangan getar layar (offset).
17	<i>(Baris Kosong)</i>	Pemisah visual antarkelas.
18	<code>class WeaponType:</code>	Deklarasi kelas statistik senjata permainan.
19	<code>UZI = {"name": "MICRO UZI (SMG)", ...}</code>	Kamus data Micro Uzi: damage 25, peluru 40, recoil ringan 6px, delay tembak sangat cepat 3 frame (beruntun).
20	<code>SCAR = {"name": "SCAR-H (ASSAULT)", ...}</code>	Kamus data SCAR-H: damage 45, peluru 30, recoil sedang 14px, delay tembak 6 frame.

21	SHOTGUN = {"name": "SPAS-12 (SHOTGUN)", ...}	Kamus data Spas-12: damage besar 120, peluru 8, recoil kuat 32px, delay tembak lambat 28 frame.
22	SNIPER = {"name": "AWM SNIPER", ...}	Kamus data AWM Sniper: damage sangat besar 250, peluru 5, recoil sangat kuat 48px, delay tembak sangat lambat 55 frame.
23	(Baris Kosong)	Pemisah visual antarkelas.
24	class TacticalWeapon:	Deklarasi kelas TacticalWeapon untuk memproses logika individual senjata aktif.
25	def __init__(self, w_type, reload_max_time=60):	Konstruktur menerima kamus tipe senjata (w_type) dan batas waktu reload default 60 frame.
26	self.type = w_type	Menyimpan kamus tipe data senjata.
27	self.ammo_max = w_type["ammo"] self.ammo, self.is_reloading, self.reload_timer, self.shot_delay, self.reload_max_time = self.ammo_max, False, 0, 0, reload_max_time	Menyimpan kapasitas amunisi maksimal senjata.
28		Mengeset peluru awal, status reload (False), timer reload (0), jeda tembak (0), dan batas maksimal waktu reload.
29	def update(self):	Memperbarui sisa jeda tembak dan durasi reload senjata per frame.
30	if self.shot_delay > 0: self.shot_delay -= 1	Jika jeda tembak aktif, kurangi nilainya sebanyak 1 tick.
31-33	if self.is_reloading: ...	Jika sedang reload: kurangi timer reload. Jika timer habis, isi peluru penuh kembali dan set status reload ke False.
34	def can_shoot(self): return self.ammo > 0 and not self.is_reloading and self.shot_delay <= 0	Mengembalikan status True jika peluru masih ada, tidak sedang reload, dan jeda tembak bernilai 0.
35	def shoot(self): self.ammo -= 1; self.shot_delay = self.type["delay"]	Mengurangi amunisi sebanyak 1 dan memasang jeda delay tembak sesuai jenis senjata.
36-37	def reload(self): ...	Memulai proses pengisian peluru baru jika sedang tidak reload dan peluru di magazine belum penuh.
38	(Baris Kosong)	Pemisah visual antarkelas.
39	class ShootingRangeUltimate(MiniGame):	Deklarasi kelas utama minigame "City Under Attack".
40-41	def __init__(self, screen, clock, manager): super().__init__(...)	Konstruktur minigame memanggil inisialisasi kelas induk MiniGame.
42	self.width, self.height = 1280, 720 self.crosshair, self.score, self.timer, self.kills, self.headshots, self.bronze_earned, self.boss_spawned_count = [640, 360], 0, 90.0, 0, 0, 0, 0	Resolusi layar minigame: 1280x720 piksel.
43		Inisialisasi kursor tengah, skor 0, sisa waktu awal 90 detik, eliminasi, headshot, perolehan Bronze, dan hitungan boss spawn.
44-47	try: self.bg_img = ... except: ...	Memuat gambar latar belakang kota hancur city_bg.png seukuran layar (fallback warna abu-abu biru gelap jika gagal).

48-52	<pre> self.weapon_images = {} ... </pre>	Loop otomatis untuk memuat 4 gambar visual model senjata FPS berskala (Micro Uzi, SCAR-H, Spas-12, AWM Sniper).
53-55	<pre> try: self.zombie_sheet = ... </pre>	Memuat spritesheet visual zombie musuh berskala pembesaran 2.5x.
56	<pre> upgrades = getattr(self.manager.main_engine, "weapon_upgrades", ...) </pre>	Mengambil setelan data upgrade level senjata dari dunia Sanctuary utama.
57-63	<pre> self.weapons = {} ... </pre>	Menginstansiasi 4 senjata pemain dengan penyesuaian bonus upgrade (Lv upgrade menambah Damage +25%, kapasitas Ammo +25%, mempercepat reload, dan mempercepat firerate).
64	<pre> self.current_weapon_name = getattr(self.manager.main_engine, "selected_weapon", "SCAR") </pre>	Sistem Senjata Terkunci: Mengunci senjata minigame agar sesuai dengan senjata pilihan terakhir di Sanctuary.
65	<pre> self.targets, self.particles, self.floating_texts, self.flash_timer, self.shake_v = [], [], [], 0, 0 </pre>	Inisialisasi list zombie aktif, partikel, teks melayang, timer kilatan tembakan, dan kekuatan getar layar aktif.
66	<pre> self.font_header, self.font_tactical = pygame.font.SysFont("monospace", 36, bold=True), pygame.font.SysFont("monospace", 18, bold=True) </pre>	Inisialisasi font teks utama 36px tebal dan font UI taktis 18px tebal.
67-70	<pre> try: self.shoot_sfx = ... </pre>	Memuat file audio efek suara tembakan senjata (suarapistol.mp3) dengan volume 60%.
71	<pre> (Baris Kosong) </pre>	Pemisah visual.
72-73	<pre> @property def weapon(self): return self.weapons[self.current_weapon_name] </pre>	Properti getter mengambil objek senjata aktif pemain saat ini.
74	<pre> (Baris Kosong) </pre>	Pemisah visual.
75	<pre> def _trigger_shot(self): </pre>	Metode internal menembakkan satu peluru dan memproses pencocokan tembakan.
76	<pre> self.weapon.shoot(); self.flash_timer, self.shake_v = 5, self.weapon.type["recoil"] </pre>	Kurangi peluru, picu kedipan flash putih 5 frame, pasang getaran recoil layar.
77	<pre> if self.shoot_sfx: self.shoot_sfx.play() </pre>	Putar efek suara tembakan.
78	<pre> hit = False </pre>	Status penanda apakah tembakan mengenai zombie (awal False).
79-80	<pre> for t in self.targets: hx, hy, hr = ... </pre>	Mengambil koordinat kepala zombie (titik tengah sumbu X, sumbu Y ber-offset ke atas, dan radius kepala 40% dari ukuran tubuh).
81-82	<pre> dist_head = math.hypot(...) ; dist_body = ... </pre>	Menghitung jarak bidikan crosshair ke kepala zombie dan jarak ke badan zombie.
83	<pre> if dist_head < hr or dist_body < t['size']: </pre>	Jika cursor masuk ke radius kepala atau radius badan zombie.
84	<pre> is_hs = dist_head < hr </pre>	Status boolean is_hs bernilai True jika mengenai kepala.

85	<pre> dmg = self.weapon.type["damage"] * (2 if is_hs else 1) </pre>	Damage tembakan dilipatgandakan menjadi 2x lipat jika Headshot.
86	<pre> self.floating_texts.append(...) </pre>	Menambahkan teks melayang kerusakan berwarna kuning emas jika "CRITICAL! Headshot" atau abu-abu jika tembakan biasa.
87	<pre> for _ in range(15 if is_hs else 8): self.particles.append(...) </pre>	Memunculkan partikel sebaran darah zombie di titik benturan peluru (lebih banyak jika headshot).
88	<pre> t['hp'] -= dmg </pre>	Mengurangi nyawa zombie.
89	<pre> if t['hp'] <= 0: </pre>	Jika nyawa zombie habis (zombie tereliminasi).
90	<pre> self.kills += 1; is_boss = t.get('is_boss', False) </pre>	Tambah total eliminasi, cek status apakah target merupakan Boss Zombie.
91	<pre> mult = 1.0 if self.kills < 10 else 1.5 if self.kills < 25 else 2.0 if self.kills < 50 else 3.0 if self.kills < 80 else 5.0 </pre>	Menghitung pengali skor beruntun (<i>Score Multiplier</i>) berdasarkan jumlah eliminasi.
92-96	<pre> if is_boss: ... </pre>	Logika kematian Boss Zombie: +1000 Poin, +200 Bronze, bonus tambahan waktu +20 detik, picu shake layar kuat (40px), dan pemicu sebaran partikel masif.
97-103	<pre> else: ... </pre>	Logika kematian zombie biasa: tambah headshot tracker jika sesuai, skor +150 (HS) / +100 (Body), klaim uang dikalikan multiplier, bonus waktu +4.0s (HS) / +2.5s (Body).
104	<pre> t['active'] = False </pre>	Menonaktifkan status zombie agar terhapus dari daftar aktif.
105	<pre> hit = True; break </pre>	Mengubah status hit ke True dan hentikan perulangan target (satu peluru menembus satu zombie terdepan).
106-107	<pre> if not hit: for _ in range(3): self.particles.append(...) </pre>	Jika peluru meleset mengenai latar belakang tembok, buat 3 partikel percikan abu-abu.
108	<pre> (Baris Kosong) </pre>	Pemisah visual.
109	<pre> def handle_event(self, event): </pre>	Metode memproses kejadian input mouse fisik dan keyboard.
110	<pre> if event.type == pygame.MOUSEMOTION: self.crosshair = list(event.pos) </pre>	Mengikuti posisi mouse jika pemain menggerakkan mouse fisik.
111-112	<pre> if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1 and self.weapon.can_shoot(): self._trigger_shot() </pre>	Tembak 1 peluru saat klik kiri mouse ditekan dan senjata siap.
113-114	<pre> if event.type == pygame.KEYDOWN: if event.key == pygame.K_r: self.weapon.reload() </pre>	Lakukan pengisian peluru manual jika tombol R ditekan.
115	<pre> (Baris Kosong) </pre>	Pemisah visual.
116	<pre> def update(self, dt): </pre>	Metode update logika minigame per frame.
117	<pre> self.timer -= dt / 60.0 </pre>	Mengurangi sisa waktu bermain minigame.

118	<pre> if self.timer <= 0: self.exit_game({'score': self.score, 'bronze_earned': self.bronze_earned}) for w in </pre>	Jika waktu habis, matikan minigame dan laporkan skor ke Sanctuary.
119	<pre> self.weapons.values(): w.update() </pre>	Memperbarui status cooldown internal di seluruh daftar senjata.
120-127	<pre> # Gesture-based reloading ... </pre>	Reload Gestur: Membaca thread gestur tangan. Jika tangan mengepal (FIST) atau gestur (RELOAD) terdeteksi dan magazine belum penuh, picu reload otomatis.
128-133	<pre> is_auto = self.current_weapon_name in ["UZI", "SCAR"] ... </pre>	Tembak Otomatis (Uzi/Scar): Jika tombol mouse ditekan terus menerus ATAU gestur tangan berbentuk pistol (PISTOL) diacungkan, tembak secara otomatis secepat mungkin sesuai jeda senjata.
134-135	<pre> if self.flash_timer > 0: self.flash_timer -= 1 </pre>	Mengurangi timer kedipan efek tembak.
136	<pre> if self.shake_v > 0: self.shake_v *= 0.85 </pre>	Meredam getaran layar secara bertahap (dikalikan 0.85 per frame).
137-141	<pre> next_boss = 15 * (self.boss_spawned_count + 1) ... </pre>	AI Melahirkan Boss: Lahirkan Boss Zombie Crimson Abomination setiap kelipatan 15 eliminasi. Pasang nyawa tebal (600 HP) dan beri getaran layar kuat.
142-146	<pre> max_targets = 8 + min(7, self.kills // 5) ... </pre>	AI Melahirkan Zombie: Melahirkan zombie baru di koordinat acak jika jumlah di layar kurang dari kapasitas maksimum target (kapasitas bertambah sulit seiring eliminasi).
147-156	<pre> speed_mult = 1.0 + (self.kills / 25.0) ... </pre>	AI Gerakan Zombie: Menggerakkan koordinat zombie (zombie bergerak turun ke bawah jika sumbu animasi bernilai 4, bergerak ke kanan jika sumbu 6, dan bergerak ke kiri jika sumbu 7). Kecepatan zombie bertambah seiring eliminasi.
157	<pre> self.targets = [t for t in self.targets if t['active'] and t['timer'] > 0 and t['pos'][1] < 670] </pre>	Menyaring dan menyisakan zombie yang masih aktif, durasinya belum habis, dan belum melintasi batas pertahanan bawah (670px).
158	<pre> self.particles = [p for p in self.particles if p.update()] for ft in </pre>	Memperbarui status partikel cipratan darah di layar.
159-160	<pre> self.floating_texts: ft['pos'][1] -= 0.8; ft['timer'] -= 1 ... </pre>	Menggeser teks melayang ke atas dan menyaring teks yang durasinya habis.
161-169	<pre> closest_t = None; min_d = 90.0 ... </pre>	Sistem Bantuan Bidik (Aim Assist): Memindai seluruh zombie terdekat. Jika koordinat kepala zombie berada di dalam radius sensitivitas bidikan (< 90px), tarik/geser kursor crosshair secara magnetis mendekat sebesar 55% agar mempermudah pemakaian tangan webcam.

170	<code>def _draw_pistol_fps(self, screen, offset):</code>	Metode menggambar model visual senjata FPS di kanan bawah layar game.
171-173	<code>cx, cy = self.crosshair ...</code>	Menghitung koordinat posisi gambar senjata agar ikut bergerak sedikit bergoyang mengikuti retikel bidikan kursor pemain.
174-177	<code>img = self.weapon_images.get(self.current_weapon_image_index) ...</code>	Menggambar visual model senjata pemain. Menampilkan sesuai recoil getaran layar.
178-180	<code>else: ...</code>	Fallback: gambar persegi panjang abu-abu sederhana sebagai model pistol darurat jika file aset gambar gagal dimuat.
181	<i>(Baris Kosong)</i>	Pemisah visual.
182	<code>def draw(self):</code>	Metode utama merender seluruh elemen visual minigame ke layar.
183-184	<code>offset = (...) ; self.screen.blit(self.bg_img, offset)</code>	Menghitung pergeseran acak getaran layar (recoil/shake) lalu merender gambar latar belakang kota.
185-197	<code>for t in self.targets: ...</code>	Loop menggambar zombie di layar. Jika spritesheet berhasil dimuat: blit frame sprite zombie (lebih besar 2.2x jika tipe Boss), gambar elips bayangan merah di tanah, dan gambar bar nyawa di atas kepala zombie.
198-199	<code>else: ...</code>	Fallback: gambar lingkaran hijau-merah sederhana sebagai visual zombie darurat jika spritesheet tidak ditemukan.
200	<code>for p in self.particles: p.draw(self.screen, offset)</code>	Menggambar seluruh partikel cipratan darah.
201-203	<code>for ft in self.floating_texts: ...</code>	Menggambar teks melayang kerusakan (dengan outline bayangan hitam agar terbaca).
204-210	<code>if self.flash_timer > 0: ...</code>	Menggambar kedipan flash layar putih kekuningan saat menembak, dan lingkaran oranye kilatan api moncong senjata (<i>muzzle flash</i>).
211	<code>self._draw_pistol_fps(self.screen, offset)</code>	Memanggil fungsi penggambaran model visual senjata FPS.
212-214	<code>pygame.draw.rect(...) ; pygame.draw.line(...)</code>	Menggambar panel kepala atas berwarna gelap dan garis pembatas neon merah setebal 3px.
215-216	<code>self.screen.blit(self.font_header.render("CITY UNDER ATTACK", ...))</code>	Menampilkan judul minigame dan total eliminasi serta pengali skor di panel atas.
217-218	<code>self.screen.blit(...)</code>	Menampilkan total skor dan perolehan Bronze di panel atas sebelah kanan.
219-220	<code>self.screen.blit(...)</code>	Menampilkan nama senjata aktif dan sisa peluru/magazine di panel atas sebelah kiri bawah.
221-224	<code>pygame.draw.rect(self.screen, (10, 10, 15, 200), (950, 600, 310, 80) ...</code>	Menggambar panel visual info senjata terkunci di pojok kanan bawah. Menampilkan teks penegasan bahwa senjata aktif saat ini sedang terkunci untuk kestabilan gameplay.
225-226	<code>mins, secs = int(self.timer)//60, int(self.timer)%60 ...</code>	Menampilkan jam pewaktu hitung mundur sisa waktu bermain minigame di bagian tengah atas.

227-234	<code>boss = next(...)</code>	Menggambar boss bar besar berwarna merah neon di bagian tengah atas jika Crimson Abomination (Boss Zombie) sedang aktif.
235-240	<code>locked = None ...</code>	Mengecek apakah target di dekat kursor terkunci ($\text{radius} < 12\text{px}$) untuk mengubah retikel menjadi merah tanda sasaran pas.
241-243	<code>pygame.draw.circle(...)</code>	Menggambar retikel kursor crosshair bidikan bulat berpendar hijau (atau merah jika target terkunci).
244	<code>if locked: pygame.draw.rect(...)</code>	Menggambar kotak target penanda merah 44x44px di tubuh zombie jika bidikan terkunci telak.
245	<i>(Baris Kosong)</i>	Akhir dari berkas kode sumber.

Mekanika Kunci Senjata & Stabilitas Gestur

- Masalah Awal:** Tangan goyang webcam sering memicu pergantian senjata yang tidak disengaja (misalpeace sign tanpa sengaja terbaca saat menembak), sehingga minigame lag dan peluru macet.
- Solusi Terkunci:** Pemilihan senjata hanya bisa dilakukan di dunia Sanctuary utama menggunakan tombol keyboard 1-4. Ketika masuk ke mode minigame, senjata aktif dikunci secara permanen (`self.current_weapon_name = selected_weapon`). Pilihan gestur webcam hanya dibatasi untuk membidik (AIM), menembak (PISTOL), dan mengisi peluru (FIST) sehingga performa minigame sangat mulus dan stabil.

■ Visualisasi Logika Gestur Pistol (PISTOL Gesture)

Dokumen ini menjelaskan visualisasi geometris dan logika pemrograman dari gestur "PISTOL" yang digunakan untuk menembak dalam game *Sanctuary Defense*.

■ Visualisasi MediaPipe Landmarks

Gambar di bawah ini mengilustrasikan titik-titik sendi tangan (*landmarks*) yang dilacak oleh MediaPipe beserta formula deteksinya:

■ Penjelasan Aturan Geometris (Rule-Based Heuristic)

Untuk mengklasifikasikan tangan sebagai gestur "PISTOL", sistem memeriksa dua kondisi utama secara bersamaan:

Kondisi	Formula Matematika	Penjelasan Teknis
Telunjuk Tegak ke Atas	$y_8 < y_6$	Koordinat Y dari Ujung Telunjuk (8) harus lebih kecil dari Sendi PIP Telunjuk (6) . Karena koordinat Y pada layar komputer dimulai dari 0 di bagian paling atas, nilai Y yang lebih kecil menandakan posisi yang lebih tinggi di dunia nyata.

Ibu Jari Terbuka/Lebar

$\text{dist}(4, 5) > 0.05$

Jarak Euclidean antara **Ujung Ibu Jari (4)** dan **Pangkal Jari Telunjuk / MCP (5)** harus lebih besar dari batas ambang 0.05 (sekitar 5% lebar/tinggi bidang tangkapan kamera). Ini memastikan jempol tidak menekuk ke dalam telapak tangan.

■ Alur Logika Pemicu Tembakan (Recoil & Cooldown)

Setelah gestur terdeteksi sebagai "PISTOL", game tidak langsung menembak secara konstan. Pemain harus menyentakkan tangannya ke atas (mensimulasikan sentakan recoil senjata) untuk menembakkan sebutir peluru.

Berikut adalah diagram alur keputusan pemicuan tembakan (*firing trigger state*):

```
graph TD
    Start([Frame Baru Terbaca]) --> DetectGesture{Apakah Gestur = 'PISTOL'?}
    DetectGesture -- Tidak --> Reset[Setel Gestur ke 'None']
    DetectGesture -- Ya --> CalcVelocity[Hitung Kecepatan Vertikal Jari Telunjuk  
velocity_y = Last_Y - Current_Y]
    CalcVelocity --> ApplyFilter[Terapkan EMA Filter  
velocity_y = alpha * raw + 1-alpha * velocity_y]
    ApplyFilter --> CheckJerk{Apakah velocity_y > 0.04?}
    CheckJerk -- Tidak --> End([Selesai / Tunggu Frame Berikutnya])
    CheckJerk -- Ya --> CheckCooldown{Apakah Jeda Waktu > 0.25 detik?}
    CheckCooldown -- Tidak --> End
    CheckCooldown -- Ya --> TriggerFire[Setel recoil_triggered = True  
Mainkan Efek Tembakan & Recoil]
    TriggerFire --> UpdateTime[Perbarui last_shot_time = Sekarang]
    UpdateTime --> End
```

[!TIP] Mengapa menggunakan filter EMA (Exponential Moving Average)? Filter ini menghaluskan grafik perubahan koordinat sehingga getaran kecil dari tangan pemain tidak terbaca secara salah sebagai gerakan sentakan menembak yang tidak disengaja.

■ Referensi Kode Terkait

- Definisi Gestur: [gestures.py#L60-L71](#)
- Deteksi Kecepatan & Recoil: [gestures.py#L168-L182](#)
- Penjelasan Dokumentasi Asli: [gestures.md](#)